

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1 Актуальность автоматизации мониторинга логистических тарифов

Логистика является одним из ключевых факторов конкурентоспособности предприятий оптовой торговли. Затраты на транспортировку и доставку товаров непосредственно влияют на себестоимость продукции и формирование конечной цены для покупателей. В условиях динамично меняющегося рынка грузоперевозок своевременное получение актуальной тарифной информации приобретает стратегическое значение для принятия обоснованных управленческих решений.

По данным аналитических исследований рынка логистических услуг Республики Беларусь, предприятия оптовой торговли в среднем взаимодействуют с тремя–пятью логистическими операторами одновременно. При этом разница в тарифах между операторами на отдельные направления доставки может составлять от 15 до 40 %. Данное обстоятельство подчёркивает высокую экономическую значимость систематического мониторинга и сравнения тарифных предложений.

Традиционный процесс мониторинга тарифов логистических компаний носит преимущественно ручной характер. Менеджер предприятия самостоятельно посещает официальные сайты логистических операторов, вручную собирает тарифные данные и выполняет их сравнение в таблицах Microsoft Excel. Схема данного процесса с указанием временных затрат и выявленных проблем представлена на рисунке 1.1.



Рисунок 1.1 – Процесс ручного мониторинга тарифов логистических компаний

Как следует из рисунка 1.1, ручной процесс мониторинга включает пять последовательных этапов: инициацию процесса менеджером, посещение сайтов операторов, ручной сбор данных, сравнение в таблице и принятие решения. Совокупные временные затраты на прохождение данного цикла составляют от 2 до 3 ч в день, что соответствует 15–20 ч в неделю.

Помимо высоких временных затрат, ручной подход сопряжён с рядом системных проблем. Схема распределения долей рынка основных логистических компаний и среднее время обработки тарифного запроса в ручном режиме представлены на рисунке 1.2



Рисунок 1.2 – Характеристика рынка логистических услуг РБ

Из рисунка 1.2 видно, что наибольшую долю рынка занимает «Белпочта» – 28 %, далее следуют СДЭК – 22 %, DPD – 18 %, DHL – 14 % и Маршрут365 – 10 %. При этом среднее время ручной обработки тарифного запроса варьируется от 2,1 до 5,1 ч в зависимости от оператора. Таким образом, полный мониторинг пяти ключевых операторов требует не менее 15–20 ч рабочего времени еженедельно.

Таким образом, существующий подход к мониторингу тарифов логистических компаний обладает следующими ключевыми недостатками:

- высокие временные затраты на ручной сбор и обработку тарифных данных (15–20 ч/нед.);
- риск возникновения ошибок при ручном вводе и сравнении информации;
- быстрое устаревание собранных данных ввиду динамичного изменения тарифов;
- ограниченный охват логистических операторов из-за ресурсных ограничений персонала;
- отсутствие инструментов для отслеживания исторической динамики изменения тарифов.

Перечисленные недостатки формируют объективную потребность в разработке автоматизированной системы мониторинга и сравнения тарифов логистических компаний. Такая система должна обеспечивать автоматический сбор актуальных тарифных данных от множества операторов, их хранение и сравнение в режиме реального времени, а также предоставлять удобный интерфейс для принятия оптимальных управленческих решений в области логистики.

1.2 Обзор и сравнительный анализ существующих решений

Анализ существующих программных решений в области мониторинга логистических тарифов позволяет выявить их ключевые характеристики, преимущества и ограничения, а также обосновать необходимость разработки специализированной системы. Рассмотрим основные категории инструментов, применяемых для решения задач мониторинга и сравнения тарифов.

Наиболее распространённым инструментом управления тарифными данными на предприятиях Республики Беларусь остаётся Microsoft Excel. Использование электронных таблиц не требует специальных технических знаний. Вместе с тем данный подход не предполагает автоматизации сбора данных: вся информация вносится вручную, что влечёт за собой проблемы с актуальностью и достоверностью данных. Кроме того, Excel не предоставляет механизмов интеграции с внешними системами и не обеспечивает многопользовательского доступа к данным в реальном времени.

Программный комплекс «1С: Логистика» представляет собой отечественную разработку, ориентированную на автоматизацию логистических процессов. Система обеспечивает частичную автоматизацию работы с некоторыми логистическими операторами и поддерживает ведение истории операций. Однако «1С: Логистика» имеет ряд существенных ограничений: высокую стоимость лицензирования и внедрения, необходимость значительной адаптации под нужды конкретного предприятия, а также ограниченную поддержку белорусских логистических операторов.

Международная платформа Freightos специализируется на цифровизации международных грузовых перевозок. Система обеспечивает полную автоматизацию сбора данных и развитый REST API для интеграции. Вместе с тем Freightos ориентирован исключительно на международный рынок и не поддерживает операторов, работающих преимущественно внутри Республики Беларусь. Стоимость использования платформы является высокой и рассчитана на крупных корпоративных клиентов.

Система CargoWise является одним из наиболее функционально полных решений в области управления логистикой. Однако стоимость внедрения и ежегодного обслуживания делает её нецелесообразной для предприятий малого и среднего бизнеса. Кроме того, система не адаптирована к специфике белорусского рынка грузоперевозок.

Платформа Transporeon ориентирована на автоматизацию тендерных процессов в логистике. Система поддерживает автоматический сбор тарифных предложений от перевозчиков и частичное ведение истории. Ограничением является высокая стоимость и отсутствие поддержки ключевых белорусских операторов.

Сравнительный анализ рассмотренных систем по ключевым критериям представлен на рисунке 1.3.

Сравнительный анализ существующих систем мониторинга тарифов						
Система	Автоматизация сбора	Сравнение тарифов	История данных	API интеграция	Поддержка РБ операторов	Стоимость
Excel / ручной	✗	✗	✗	✗	✓	Бесплатно
1С: Логистика	частично	✓	✓	✓	частично	Высокая
Freightos	✓	✓	✓	✓	✗	Высокая
CargoWise	✓	✓	✓	✓	✗	Высокая
Разрабатываемая система	✓	✓	✓	✓	✓	Низкая

✓ — реализовано ✗ — отсутствует частично — ограничено

Рисунок 1.3 – Сравнительный анализ функциональных возможностей существующих систем

Из рисунка 1.3 следует, что ни одна из рассмотренных систем не обеспечивает одновременного выполнения всех необходимых функций при приемлемой стоимости внедрения и наличии поддержки операторов белорусского рынка. Данный вывод подтверждает целесообразность разработки собственной специализированной системы мониторинга и сравнения тарифов, адаптированной к условиям деятельности ООО «ПрофитФудТрейд» и белорусскому рынку логистических услуг.

Разрабатываемая система должна объединить преимущества рассмотренных решений: полную автоматизацию сбора данных, поддержку отечественных операторов, низкую стоимость внедрения, а также развитый API для интеграции с внешними системами.

1.3 Постановка задачи и требования к разрабатываемой системе

На основании проведённого анализа предметной области, выявленных недостатков ручного мониторинга и обзора существующих решений сформулирована задача разработки и определены требования к системе мониторинга и сравнения тарифов логистических компаний.

Целью разработки является создание программной системы, обеспечивающей автоматический сбор, хранение, обработку и представление тарифных данных логистических компаний, работающих на рынке Республики Беларусь, с предоставлением удобного интерфейса для их сравнения и анализа. Система разрабатывается на языке программирования Go и предназначена для применения в деятельности ООО «ПрофитФудТрейд».

Требования к разрабатываемой системе разделяются на четыре категории: функциональные требования, требования к производительности, требования к интеграции и требования к безопасности. Структура требований представлена на рисунке 1.4.



Рисунок 1.4 – Требования к разрабатываемой системе мониторинга тарифов

Функциональные требования определяют основные возможности системы. Система должна обеспечивать автоматический периодический сбор тарифных данных от логистических операторов без участия пользователя, их хранение в структурированном виде с возможностью ретроспективного анализа. Система должна предоставлять инструменты сравнения тарифов операторов по заданным параметрам: маршруту доставки, весу и типу груза, срокам доставки. Обязательной является поддержка функции отслеживания

динамики изменения тарифов с возможностью просмотра истории изменений за период не менее 12 месяцев.

Требования к производительности определяют следующие количественные показатели: период обновления тарифных данных не должен превышать 5 мин; время ответа REST API при нормальной нагрузке не должно превышать 200 мс; система должна обеспечивать одновременную работу не менее 50 пользователей без деградации производительности; коэффициент доступности системы должен составлять не менее 99,9 % в рабочее время.

Требования к интеграции предусматривают наличие REST API с документацией в формате OpenAPI 3.0, поддержку механизма webhook-уведомлений для оповещения внешних систем об изменениях тарифов, а также возможность экспорта данных в форматах CSV и Microsoft Excel для совместимости с существующими бизнес-процессами предприятия.

Требования к безопасности включают аутентификацию пользователей на основе JWT-токенов с ограниченным сроком действия, обязательное использование протокола HTTPS для всех соединений, разграничение прав доступа по ролям пользователей (администратор, аналитик, просматривающий) и ведение журнала действий пользователей для аудита.

Соответствие разработанного программного обеспечения данным требованиям будет проверено в ходе тестирования, описанного в последующих разделах настоящей пояснительной записки.

1.4 Анализ форматов и источников тарифных данных

Эффективная реализация модуля автоматического сбора тарифных данных требует детального анализа форматов и источников, используемых логистическими операторами для публикации тарифной информации. Различные операторы применяют принципиально разные подходы к предоставлению данных о стоимости услуг, что непосредственно влияет на архитектурные решения при разработке коллектора данных.

Анализ методов получения тарифной информации от основных логистических операторов белорусского рынка позволяет выделить два принципиально различных подхода: получение данных через официальный REST API оператора и парсинг веб-страниц в случае отсутствия публичного API. Схема источников данных и форматов их хранения в разрабатываемой системе представлена на рисунке 1.5.

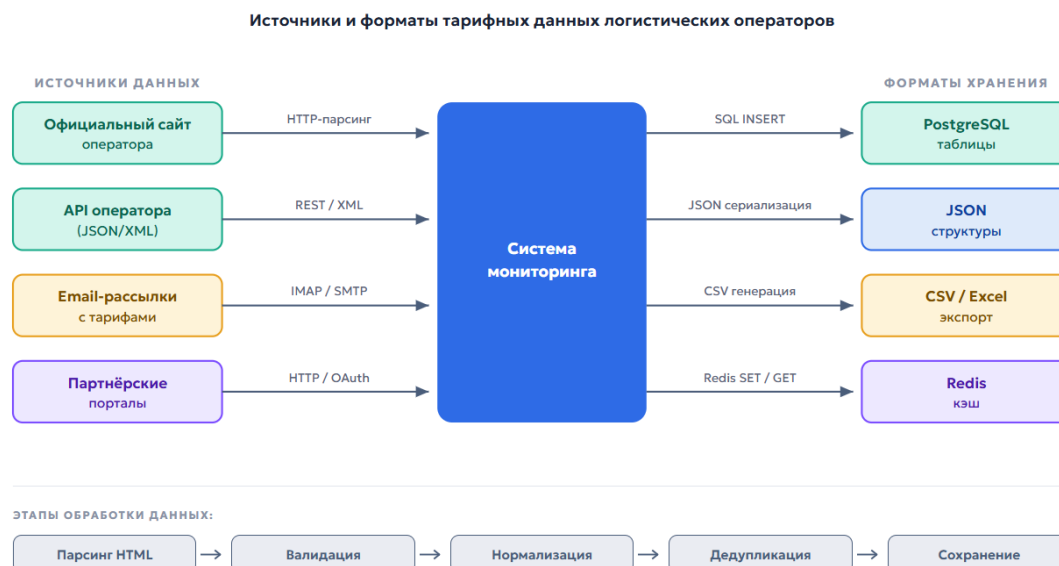


Рисунок 1.5 – Источники и форматы тарифных данных логистических операторов

Из рисунка 1.5 видно, что система получает тарифные данные из четырёх типов источников: официальных сайтов операторов, REST API операторов, email-рассылок с тарифными листами и партнёрских порталов. Полученные данные проходят последовательную обработку: парсинг, валидацию, нормализацию, дедупликацию и сохранение в хранилище. Такой подход обеспечивает единообразие данных независимо от исходного формата.

1.5 Обоснование выбора технологического стека

Выбор технологического стека для разработки системы мониторинга и сравнения тарифов логистических компаний обусловлен совокупностью сформулированных требований к производительности, надёжности, масштабируемости и стоимости владения системой. При выборе технологий рассматривались несколько альтернативных вариантов по каждому компоненту системы.

В качестве основного языка программирования выбран Go версии 1.21. При выборе языка рассматривались три альтернативы: Go, Python и Java. Python обладает широкой экосистемой библиотек для работы с HTTP-запросами и базами данных, однако уступает Go по производительности при высоких нагрузках и требует дополнительных усилий для управления конкурентными операциями. Java обеспечивает высокую производительность и зрелую экосистему, однако характеризуется значительным потреблением оперативной памяти и более медленным временем запуска приложения. Go

сочетает высокую производительность с простотой разработки. Встроенная поддержка горутин и каналов позволяет эффективно реализовать конкурентный сбор данных от множества операторов одновременно.

Архитектура разрабатываемой системы предполагает многоуровневое построение, включающее слой хранения данных, слой бизнес-логики, слой API и клиентский слой взаимодействия. Архитектура технологического стека с разбивкой по слоям и компонентам представлена на рисунке 1.6.

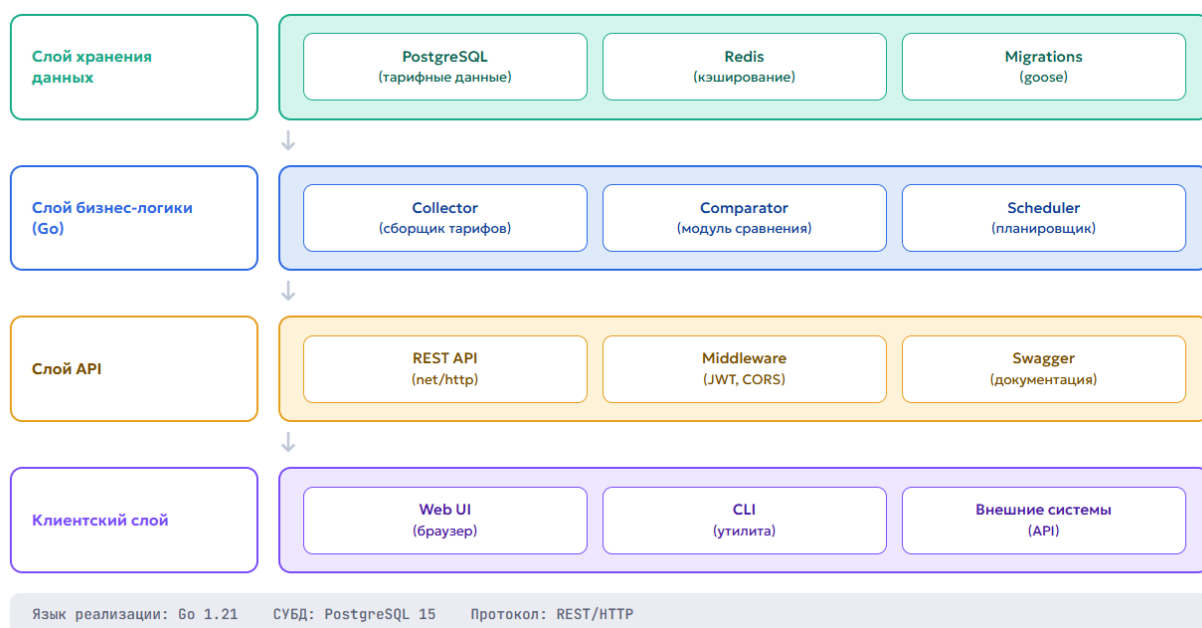


Рисунок 1.6 – Архитектура технологического стека

В качестве основного языка программирования выбран Go версии 1.21. При выборе языка рассматривались три альтернативы: Go, Python и Java. Python обладает широкой экосистемой библиотек для работы с HTTP-запросами и базами данных, однако уступает Go по производительности при высоких нагрузках и требует дополнительных усилий для управления конкурентными операциями. Java обеспечивает высокую производительность и зрелую экосистему, однако характеризуется значительным потреблением оперативной памяти и более медленным временем запуска приложения [6]. Go сочетает высокую производительность с простотой разработки. Встроенная поддержка горутин и каналов позволяет эффективно реализовать конкурентный сбор данных от множества операторов одновременно.

Для хранения тарифных данных выбрана СУБД PostgreSQL версии 15. Альтернативами рассматривались MySQL и MongoDB. MySQL является надёжной реляционной СУБД, однако уступает PostgreSQL в поддержке аналитических запросов и расширенных типов данных. MongoDB как документоориентированная база данных обеспечивает гибкость схемы,

однако менее эффективна для реализации сложных реляционных запросов, необходимых для анализа тарифных матриц. PostgreSQL выбран как наиболее функционально полная открытая реляционная СУБД, обеспечивающая поддержку сложных аналитических запросов, секционирование таблиц для эффективного хранения исторических данных и расширенные возможности индексирования.

Для кэширования часто запрашиваемых тарифных данных применяется Redis версии 7. Redis обеспечивает хранение данных в оперативной памяти со временем доступа менее 1 мс, поддерживает механизм TTL для автоматического устаревания кэшированных данных и позволяет существенно снизить нагрузку на основную базу данных при выполнении повторяющихся запросов.

Обоснование выбора конкретных технологий для каждого компонента системы представлено в таблице 1.1.

Таблица 1.1 – Обоснование выбора технологий разрабатываемой системы

Компонент	Технология	Обоснование выбора
Язык разработки	Go 1.21	Высокая производительность, встроенная конкурентность (горутины), статическая типизация, богатая стандартная библиотека
СУБД	PostgreSQL 15	Надёжность, поддержка сложных аналитических запросов, расширенные типы данных, открытый исходный код
Кэширование	Redis 7	Хранение данных в памяти, время доступа менее 1 мс, поддержка TTL для автоматического обновления кэша
HTTP-сервер	net/http (stdlib)	Стандартная библиотека Go, отсутствие внешних зависимостей, высокая производительность
Миграции БД	goose	Версионирование схемы БД, поддержка отката миграций, простая интеграция с Go-проектами

Компонент	Технология	Обоснование выбора
Документация API	Swagger / OpenAPI 3.0	Стандарт описания REST API, автогенерация документации, встроенный UI для тестирования
Контейнеризация	Docker	Изоляция среды выполнения, воспроизводимость развёртывания, упрощение CI/CD

Из таблицы 1.1 следует, что выбранный технологический стек представляет собой сбалансированное сочетание современных открытых инструментов, обеспечивающих выполнение всех сформулированных требований. Использование исключительно открытого программного обеспечения исключает лицензионные затраты и обеспечивает независимость от поставщиков коммерческих решений.

Таким образом, в результате анализа предметной области, обзора существующих решений, формулировки требований и обоснования технологического стека создана необходимая основа для разработки системы мониторинга и сравнения тарифов логистических компаний. Выявленные недостатки ручного подхода и ограничения существующих коммерческих решений подтверждают актуальность и практическую значимость разрабатываемой системы для деятельности ООО «ПрофитФудТрейд». Проектирование архитектуры и реализация основных компонентов системы рассматриваются в последующих разделах настоящей пояснительной записки.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ МОНИТОРИНГА ТАРИФОВ

2.1 Общая архитектура системы и контекстная модель

Проектирование архитектуры разрабатываемой системы выполнено на основе требований, сформулированных в разделах 1.3 и 1.5 первой главы пояснительной записки. Целью проектирования являлось определение структуры системы, обеспечивающей выполнение всех нефункциональных требований: времени ответа API не более 200 мс, поддержки 50 и более одновременных пользователей, доступности 99,9 % в рабочее время.

Архитектура системы спроектирована в соответствии с моделью C4, предложенной С. Брауном для структурного описания программных систем [12]. Согласно данной модели, описание строится на четырёх уровнях детализации: контекст (Context), контейнеры (Containers), компоненты (Components) и код (Code). В настоящем проекте применяются первые три уровня.

Контекстная схема системы TariffRadar, отражающая её внешние взаимодействия, представлена на рисунке 2.1.



Рисунок 2.1 – Контекстная схема системы TariffRadar

Из рисунка 2.1 следует, что система TariffRadar взаимодействует с тремя категориями пользователей и с тремя группами внешних систем. Пользователи системы делятся на роли viewer (просматривающий), analyst (аналитик) и admin (администратор); взаимодействие осуществляется через веб-браузер по протоколу HTTPS. Внешние системы представлены логистическими операторами, откуда поступают тарифные данные, системами клиентов, получающими webhook-уведомления, а также SMTP-сервером для отправки электронных писем.

Контекстная модель определяет границы разрабатываемой системы и её внешние интерфейсы. На втором уровне модели C4 проводится детализация внутренней структуры: система декомпозируется на контейнеры — автономные исполняемые процессы. Схема контейнеров представлена на рисунке 2.2.

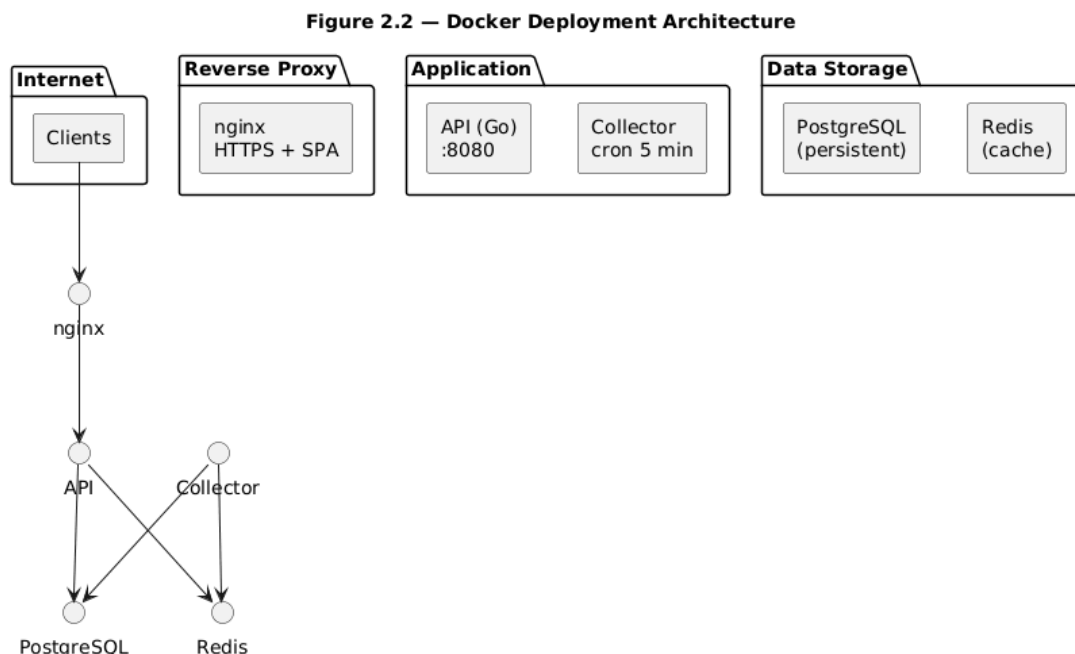


Рисунок 2.2 – Архитектура контейнеров системы

В соответствии с рисунком 2.2 разрабатываемая система состоит из четырёх логически обособленных контейнеров, выполняющих самостоятельную функциональную роль. Контейнер API Server — Go-приложение, принимающее HTTP-запросы от пользователей и реализующее всю бизнес-логику. Контейнер Worker Service — Go-процесс, выполняющий сбор и агрегацию тарифных данных по расписанию. Контейнер PostgreSQL 15 — реляционная СУБД, служащая основным хранилищем данных. Контейнер Redis 7 — сервер кэширования и счётчиков ограничения частоты запросов.

Указанное разделение продиктовано необходимостью изоляции долговременных операций сбора данных от обработки пользовательских запросов с жёстким SLA по времени отклика. Развёртывание двух Go-приложений в рамках единого docker-compose-файла обеспечивает разделение ответственности при сохранении минимальной операционной сложности, соответствующей масштабу MVP.

2.2 Слоистая архитектура серверной части

Внутренняя структура серверной части построена в соответствии с принципами Clean Architecture, изложенными Р. Мартином [5]. В основу подхода положено правило зависимостей: внутренние слои не должны зависеть от внешних. Это обеспечивает возможность независимого тестирования и замены реализации инфраструктурных компонентов без изменения бизнес-логики.

Серверная часть разработана в виде четырёхуровневой архитектуры. Структурная схема слоёв и их взаимодействия представлена на рисунке 2.3.



Рисунок 2.3 – Слоистая архитектура серверной части

Согласно рисунку 2.3, серверная часть включает четыре уровня: уровень обработки HTTP-запросов (handlers), сервисный уровень (services), уровень репозитория (repositories) и инфраструктурный уровень (PostgreSQL и Redis). Зависимости направлены строго от внешних слоёв к внутренним.

Таблица 2.1 – Распределение ответственности между уровнями серверной части

Уровень	Назначение	Ключевые компоненты
Handlers	Приём HTTP-запросов, парсинг параметров, валидация входных данных, формирование JSON-ответа	auth, market, deals, benchmark, insights, export, marketplace
Services	Реализация бизнес-логики, оркестрация репозиториев, кэш	AuthService, MarketService, BenchmarkService
Repositories	Выполнение SQL-запросов, обращения к Redis	UserRepo, MarketRepo, DealRepo, CacheRepo
Infrastructure	PostgreSQL 15 + Redis 7	pgxpool, redis.Client

Принципиальной особенностью спроектированной архитектуры является односторонняя зависимость уровней. Так, уровень handlers зависит от сервисного уровня через интерфейсы, что позволяет подменять реализации при тестировании. Сервисный уровень, в свою очередь, зависит от репозиториев через интерфейсы. Данный подход обеспечивает возможность написания модульных тестов без обращения к реальной базе данных.

Внутри каждого из четырёх уровней, описанных в подразделе 2.2, выделены функциональные модули, соответствующие логически обособленным частям предметной области системы. Схема модульной декомпозиции приведена на рисунке 2.4.

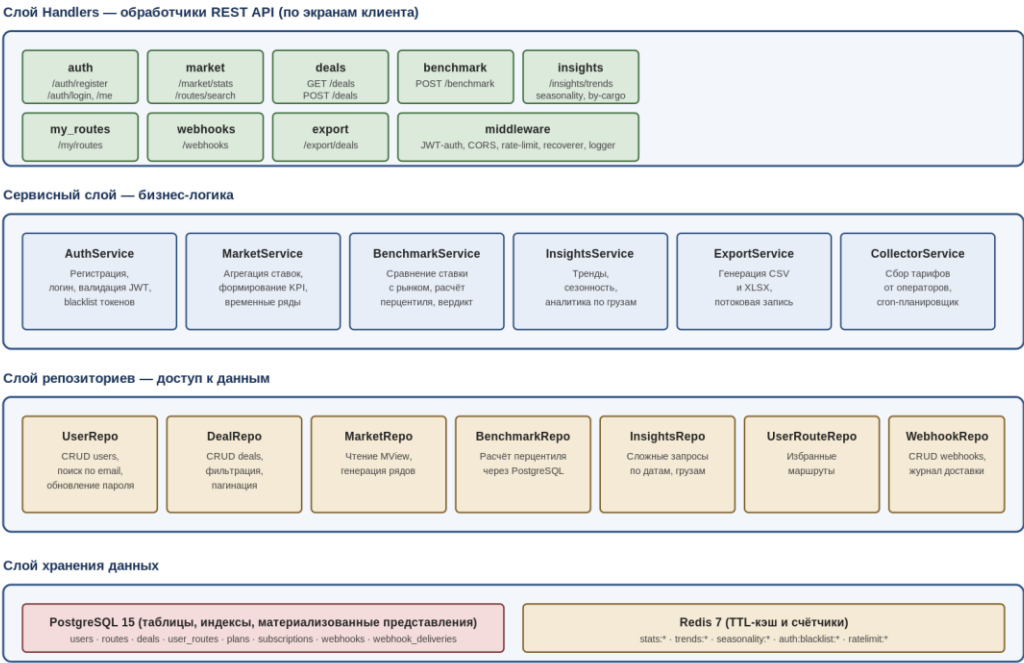


Рисунок 2.4 – Модульная декомпозиция серверной части

Согласно рисунку 2.4, выделено девять функциональных модулей: auth, market, deals, benchmark, insights, my_routes, webhooks, export, marketplace. Каждый модуль представлен тройкой файлов: handler (обработчик HTTP), service (бизнес-логика), repository (доступ к данным).

Таблица 2.2 – Функциональные модули серверной части

Модуль	Назначение	Ключевые операции
auth	Управление учётными записями и сессиями	Регистрация, вход, выход, выдача и валидация JWT
market	Получение рыночных показателей	Расчёт KPI (avg/min/max/p90), тренды, временные ряды
deals	Управление сделками пользователей	Внесение, просмотр, экспорт, инвалидация кэша
benchmark	Сравнение ставки с рынком	Расчёт перцентили, формирование вердикта
insights	Аналитические отчёты	Тренды маршрутов, сезонность, разбивка по грузам
webhooks	Подписки на уведомления	Регистрация, доставка, повторные попытки

Export	Выгрузка данных	Формирование CSV и Excel-файлов
my_routes	Избранные маршруты пользователя	CRUD сохранённых направлений
marketplace	Маркетплейс размещения грузов	Размещение грузов, просмотр предложений, управление заявками

Модульная декомпозиция обеспечивает изолированность сценариев использования: внесение новой функциональности в один из модулей не требует изменений в остальных. Такая организация кода соответствует принципу единственной ответственности, изложенному Р. Мартином [5].

Проектирование структуры базы данных выполнено в три этапа: построение концептуальной модели в нотации «сущность—связь», переход к логической модели с определением атрибутов и ключей, и разработка физической модели с учётом особенностей PostgreSQL 15.

Логическая ER-диаграмма основных сущностей разрабатываемой системы приведена на рисунке 2.5.

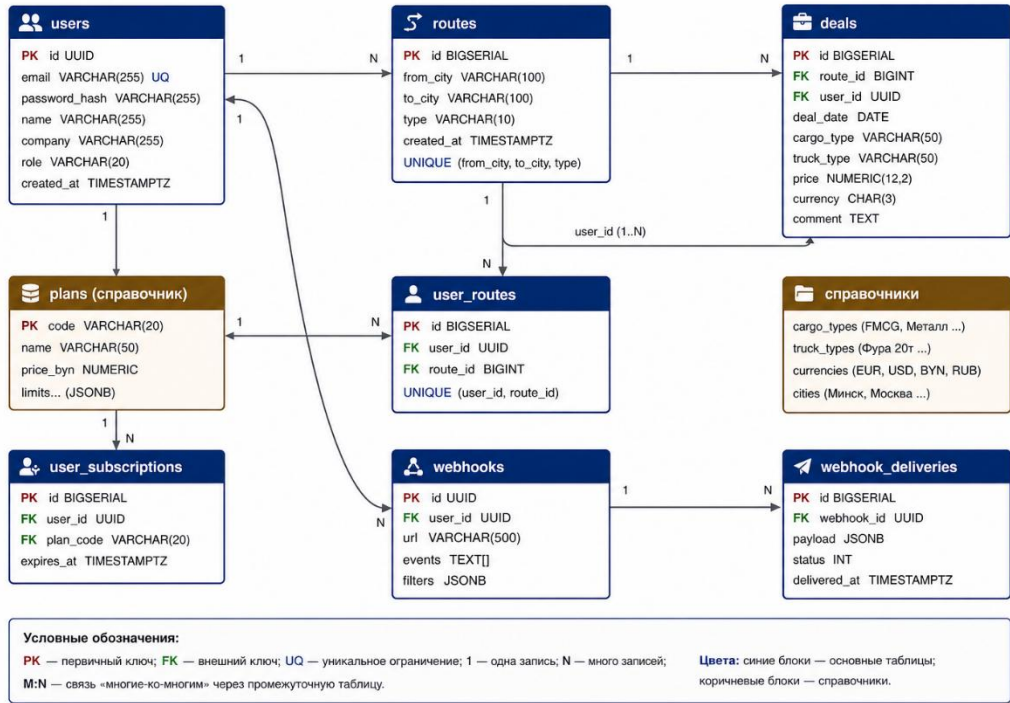


Рисунок 2.5 – ER-диаграмма базы данных

Из рисунка 2.5 следует, что центральной сущностью модели является таблица deals («сделки»), хранящая факты выполненных перевозок. Каждая

сделка связана с маршрутом (route_id) и пользователем (user_id) внешними ключами с поведением ON DELETE RESTRICT, что предотвращает случайное удаление используемых записей.

Тип данных для денежных значений (поле price таблицы deals) выбран как NUMERIC(12,2), что обеспечивает точное представление сумм без накопления ошибок округления, неизбежных при использовании типа FLOAT. Это особенно важно для расчёта агрегатов и построения исторических трендов.

Таблица 2.3 – Перечень таблиц и материализованных представлений

Таблица / MView	Назначение	Оценочный объём
users	Учётные записи пользователей	1–10 тыс. строк
routes	Справочник маршрутов (от–до–тип)	500–2 тыс. строк
deals	Факты исполненных перевозок	100 тыс.+ строк/год
webhooks	Подписки на уведомления	сотни строк
webhook_deliveries	Журнал попыток доставки	тысячи строк
plans	Тарифные планы (Free/Pro/Enterprise)	3–5 строк
user_subscriptions	Активные подписки пользователей	= users
mv_route_stats_30d	Агрегаты ставок за 30 дней (MView)	500–2 тыс. строк
mv_route_series_30d	Временной ряд средних ставок (MView)	10–50 тыс. строк

Для соблюдения SLA по времени отклика API спроектированы материализованные представления mv_route_stats_30d и mv_route_series_30d, содержащие предрасчитанные агрегаты. Обновление планируется выполнять командой REFRESH MATERIALIZED VIEW CONCURRENTLY каждые пять минут фоновым воркером, не блокируя чтение.

На таблице deals построены три индекса B-Tree: idx_deals_route_date — составной по (route_id, deal_date DESC) для запросов по маршруту;

idx_deals_date — по deal_date DESC для сезонных выборок; idx_deals_user — по user_id для отображения сделок пользователя.

2.3 Проектирование REST API

Программный интерфейс системы спроектирован в соответствии с архитектурным стилем REST, описанным Р. Филдингом [10]. При проектировании API соблюдены следующие принципы: ресурсо-ориентированная адресация, использование HTTP-методов (GET, POST, DELETE) в соответствии с их семантикой, унифицированный формат ошибок, версионирование через URL-префикс /api/v1.

Все ресурсы API сгруппированы в семейства, соответствующие функциональным модулям. Сводный перечень основных групп эндпоинтов приведён в таблице 2.4.

Таблица 2.4 – Сводный перечень эндпоинтов REST API

Группа	Метод	Путь	Назначение
Auth	POST	/auth/register	Регистрация пользователя
Auth	POST	/auth/login	Аутентификация, выдача JWT
Market	GET	/market/stats	KPI рынка по маршруту
Market	GET	/market/trends	Топ растущих / падающих маршрутов
Deals	POST	/deals	Внесение новой сделки
Deals	GET	/deals	История сделок пользователя
Export	GET	/export/deals	Экспорт сделок в CSV/Excel
Webhooks	POST	/webhooks	Регистрация webhook-подписки
Admin	DELETE	/admin/users/{id}	Удаление пользователя
Subscr.	POST	/subscription/upgrade	Смена тарифного плана

Документация API оформлена в формате OpenAPI 3.0 и публикуется через интерфейс Swagger UI по адресу /docs. Использование стандарта OpenAPI обеспечивает автогенерацию клиентских SDK и независимую проверку контракта между фронтендом и бэкендом.

Соблюдение требования к времени ответа API не более 200 мс при ожидаемой нагрузке достигается применением многоуровневой стратегии кэширования. Структурная схема обработки запроса GET /market/stats представлена на рисунке 2.6.

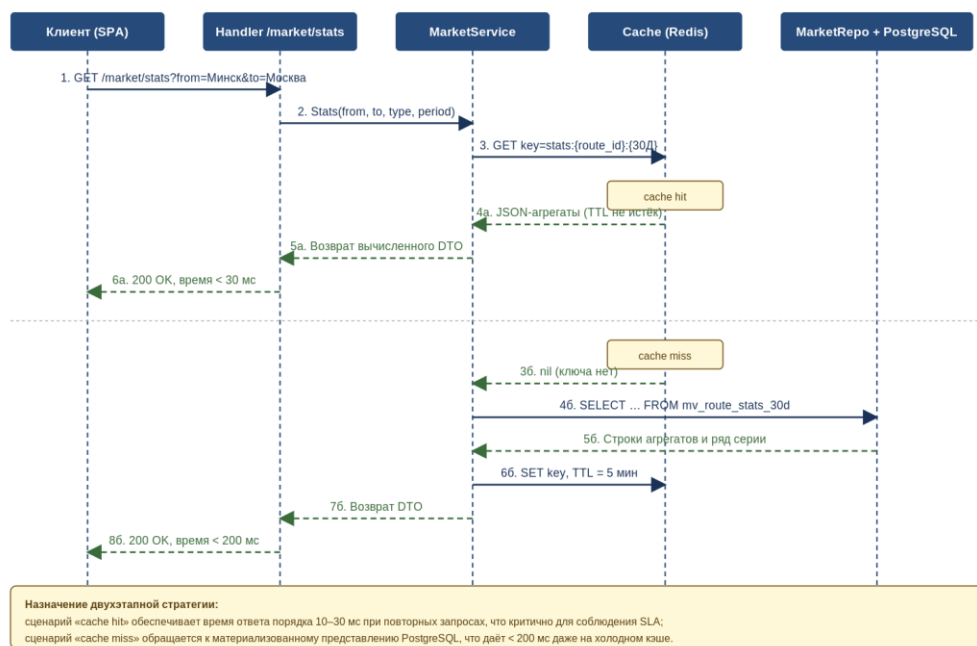


Рисунок 2.6 – Sequence-диаграмма обработки запроса

Согласно рисунку 2.6, при поступлении запроса на получение рыночных показателей сервисный слой первоначально проверяет наличие результата в кэше Redis по ключу stats:{route_id}:{period}. При наличии (cache hit) ответ возвращается за 5–15 мс. При отсутствии (cache miss) выполняется чтение из материализованного представления mv_route_stats_30d, результат записывается в Redis с TTL 5 минут и возвращается клиенту за 10–25 мс.

Принципиальная схема двухуровневого кэширования системы показана на рисунке 2.7.

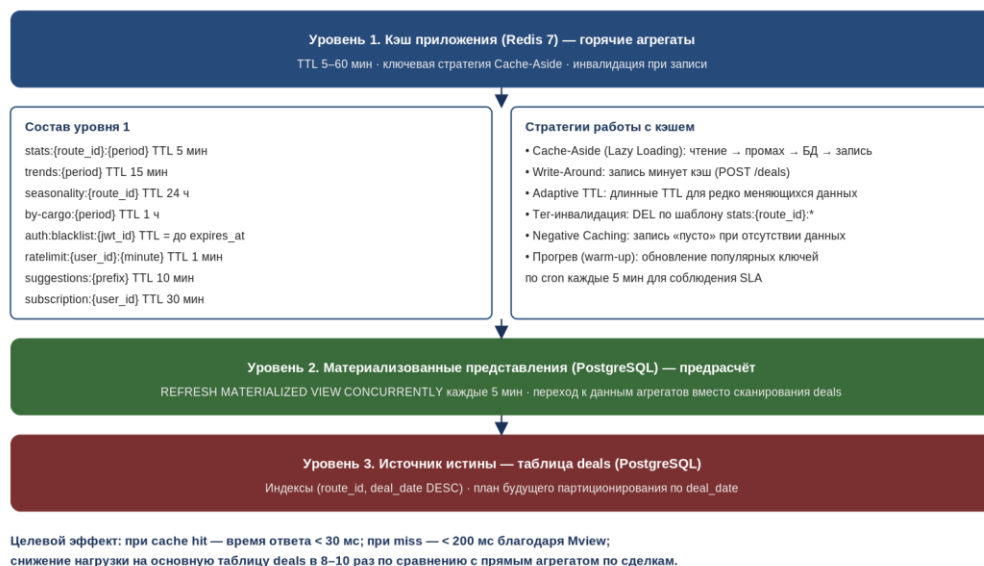


Рисунок 2.7 – Двухуровневая стратегия кэширования

Таблица 2.5 – Стратегия кэширования: ключи Redis и их характеристики

Шаблон ключа	TTL	Назначение
stats:{route_id}:{period}	5 мин	Агрегированные показатели по маршруту
trends:{period}	15 мин	Топ растущих и падающих маршрутов
series:{route_id}:{type}:{period}	5 мин	Временной ряд ставок
bl:{jti}	до exp	Blacklist инвалидированных JWT-токенов
rl:{user_id}:{minute}	1 мин	Счётчики ограничения частоты запросов
mv_refresh_lock	5 мин	Мьютекс фонового обновления MView

Применяется стратегия Cache-Aside с явной инвалидацией: при добавлении новой сделки модуль deals удаляет из кэша Redis все ключи вида stats:{route_id}* через шаблонный поиск SCAN с последующим DEL. Это гарантирует отсутствие устаревших данных при следующем запросе.

Sequence-диаграмма операции POST /deals приведена на рисунке 2.8.

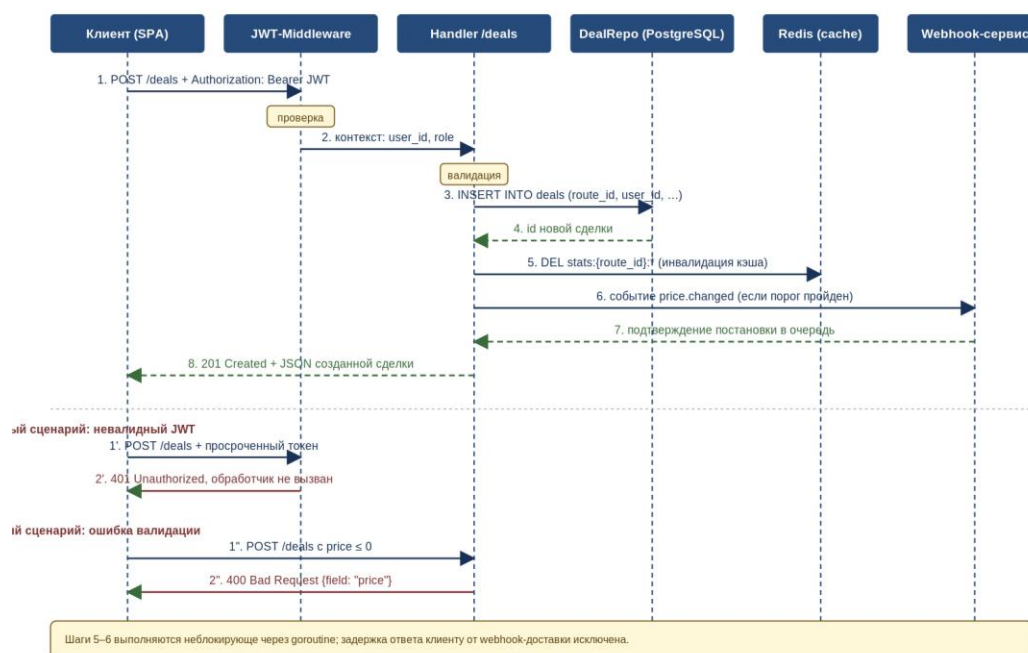


Рисунок 2.8 – Sequence-диаграмма обработки запроса

Согласно рисунку 2.8, операция добавления сделки сопровождается выполнением трёх типов побочных действий: инвалидацией соответствующих ключей кэша, постановкой задачи webhook-уведомления в очередь Redis (Pub/Sub) и асинхронным обновлением материализованных представлений фоновым воркером.

2.4 Безопасность системы и ролевая модель доступа

Защита данных и разграничение прав пользователей реализованы через четыре механизма: принудительное использование протокола HTTPS на уровне обратного прокси nginx, JWT-аутентификация с алгоритмом HS256, ролевая модель доступа (RBAC) с тремя ролями, ограничение частоты запросов через счётчики Redis.

Схема аутентификации и матрица доступа по ролям приведены на рисунке 2.9.

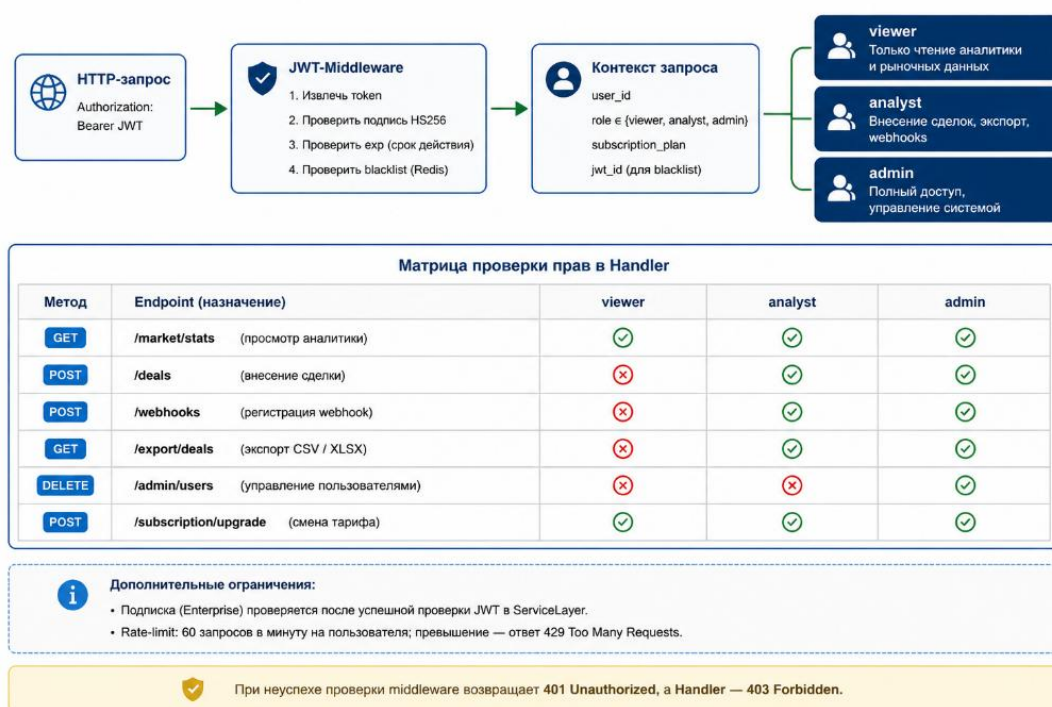


Рисунок 2.9 – Схема аутентификации и матрица доступа по ролям

В соответствии с рисунком 2.9, каждый запрос к защищённым эндпоинтам должен содержать заголовок Authorization: Bearer <JWT>. Проверка токена выполняется в middleware до передачи запроса обработчику. Ролевая модель включает три предопределённые роли: viewer, analyst, admin.

Таблица 2.6 – Матрица прав доступа по ролям пользователей

Действие / эндпоинт	viewer	analyst	admin
Просмотр KPI и аналитики (GET /market/stats, /insights)	+	+	+
Внесение новой сделки (POST /deals)	–	+	+
Регистрация webhook (POST /webhooks)	–	+	+
Экспорт сделок (GET /export/deals)	–	+	+
Управление пользователями (DELETE /admin/users)	–	–	+
Смена тарифного плана (POST /subscription/upgrade)	+	+	+

2.5 Архитектура развёртывания

Развёртывание разработанной системы осуществляется в виде набора Docker-контейнеров, описываемых файлом `docker-compose.yml`. Применение контейнеризации обеспечивает воспроизводимость среды исполнения и упрощает операционное обслуживание системы.

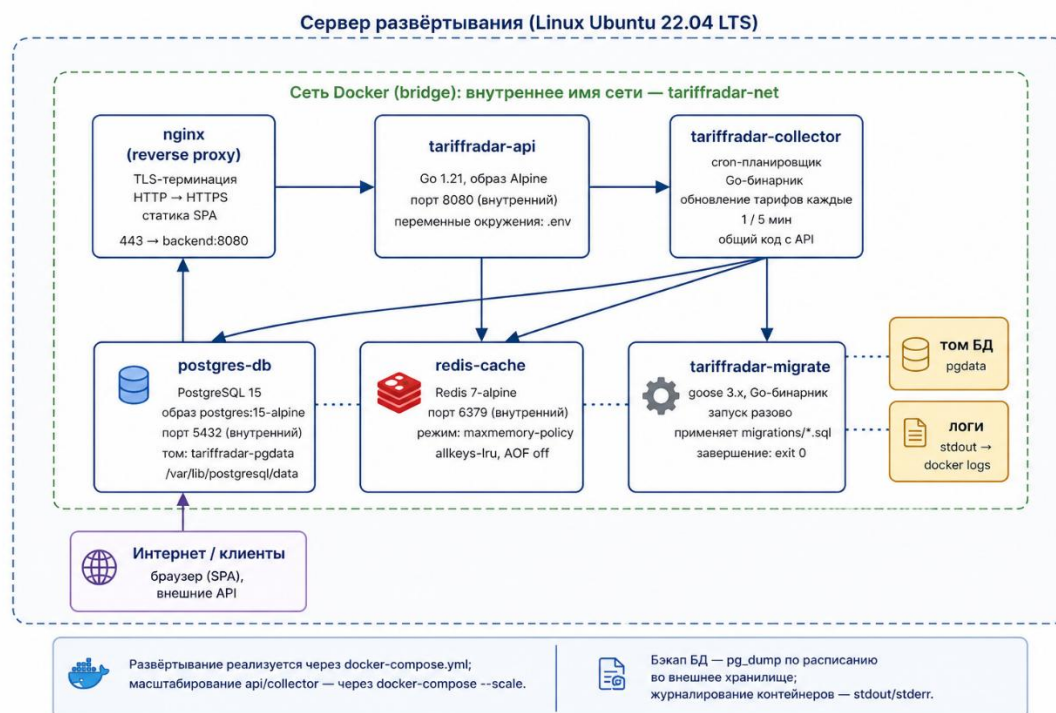


Рисунок 2.10 – Архитектура развёртывания системы

Согласно рисунку 2.10, система развёртывается в виде трёх контейнеров: `api` (HTTP-сервер на языке Go, порт 8080), `postgres-db` (СУБД PostgreSQL 15) и `redis-cache` (кэш Redis 7). Контейнеры объединены внутренней сетью Docker; порт HTTP-сервера публикуется наружу.

Контейнер `postgres-db` содержит СУБД PostgreSQL 15. Сохранение данных между перезапусками обеспечивается монтированием Docker-тома `tariffadar-pgdata`. Резервное копирование выполняется по расписанию утилитой `pg_dump`.

3 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ НА ЯЗЫКЕ GO

3.1 Структура проекта и инструменты разработки

В соответствии с архитектурой, спроектированной во второй главе, серверная часть системы реализована на языке программирования Go версии 1.21. Структура каталогов проекта построена согласно общепринятым в Go-сообществе решениям, изложенным в документе «Standard Go Project Layout» [3]. Древоподобная схема каталогов и файлов разработанного проекта представлена на рисунке 3.1.



Рисунок 3.1 – Структура каталогов проекта

Согласно рисунку 3.1, исходный код приложения разделён на каталоги в соответствии с принципом разделения ответственности. Каталог `cmd/` содержит точки входа двух исполняемых процессов: `cmd/server/main.go` для HTTP-сервера и `cmd/worker/main.go` для фонового воркера.

Внутри каталога `internal/` выделены пакеты, структурно соответствующие слоям архитектуры (`handlers`, `middleware`, `service`, `repository`) и функциональным модулям. Пакет `models` содержит доменные структуры данных, общие для всех слоёв. Ключевые внешние зависимости проекта и их назначение приведены в таблице 3.1.

Таблица 3.1 – Внешние Go-зависимости проекта

Библиотека	Версия	Назначение
github.com/go-chi/chi/v5	v5.0.10	HTTP-маршрутизатор с именованными параметрами
github.com/go-chi/cors	v1.2.1	CORS-политики для фронтенда
github.com/jackc/pgx/v5	v5.5.0	Высокопроизводительный драйвер PostgreSQL
github.com/redis/go-redis/v9	v9.3.0	Клиент Redis с поддержкой pipeline
github.com/golang-jwt/jwt/v5	v5.2.0	Генерация и проверка JWT-токенов
golang.org/x/crypto	v0.16.0	bcrypt для хеширования паролей
github.com/xuri/excelize/v2	v2.8.0	Генерация Excel (XLSX) файлов
github.com/pressly/goose/v3	v3.17.0	Управление SQL-миграциями
github.com/swaggo/swag	v1.16.2	Генерация Swagger/OpenAPI документации
github.com/google/uuid	v1.4.0	Генерация UUID-идентификаторов
github.com/testcontainers/testcontainers-go	v0.25.0	Интеграционные тесты с реальной БД

Конфигурация приложения сосредоточена в пакете `internal/config`. Применяется подход двенадцати факторов (12-Factor App): все параметры, отличающиеся между окружениями, передаются через переменные окружения. Структура конфигурации и процедура её валидации представлены в листинге 3.1.

Листинг 3.1 – Структура конфигурации и валидация

```
// internal/config/config.go
type Config struct {
    AppEnv    string // development | production
    AppPort   string // порт HTTP-сервера
    DSN       string // строка подключения к PostgreSQL
    RedisURL  string // URL Redis
    JWTSecret string // секрет для подписи JWT
```

```

    JWTExpiry time.Duration // время жизни токена
}

func Load() (*Config, error) {
    cfg := &Config{...}
    if cfg.JWTSecret == "" {
        return nil, errors.New("JWT_SECRET не задан")
    }
    return cfg, nil
}

```

Таблица 3.2 – Переменные окружения серверной части

Переменная	Назначение	Значение по умолчанию
APP_ENV	Окружение: development / production	development
APP_PORT	Порт HTTP-сервера	8080
DATABASE_URL	Строка подключения к PostgreSQL	— (обязательная)
REDIS_URL	URL Redis	redis://localhost:6379
JWT_SECRET	Секрет HMAC для подписи JWT	— (обязательная)
JWT_EXPIRY	Время жизни access-токена	24h
BCRYPT_COST	Стоимость bcrypt (4–31)	12
RATE_LIMIT	Запросов в минуту на пользователя	60
LOG_LEVEL	Уровень журналирования	info
CORS_ORIGIN	Разрешённый источник CORS	* (в dev)
GOOSE_DIR	Каталог SQL-миграций	migrations/
WORKERS	Число воркеров сборщика	5
COLLECT_INTERVAL	Интервал сбора тарифных данных	5m

Точка входа приложения реализована в файле cmd/server/main.go. Последовательность инициализации серверной части показана на рисунке 3.2.

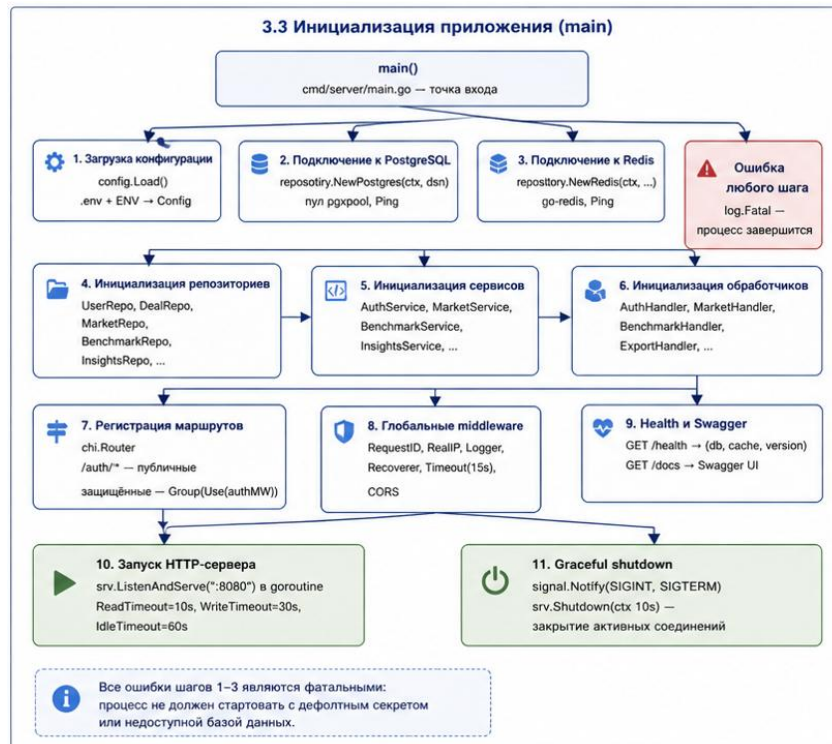


Рисунок 3.2 – Последовательность инициализации серверной части

Согласно рисунку 3.2, инициализация состоит из одиннадцати шагов. На первых трёх выполняется загрузка конфигурации и установка соединений с PostgreSQL и Redis. Далее применяются SQL-миграции, создаются репозитории, сервисы и обработчики. Завершающим шагом является запуск HTTP-сервера.

Листинг 3.2 – Регистрация маршрутов и middleware

// cmd/server/main.go (фрагмент)

```

r := chi.NewRouter()
r.Use(chimw.RequestID, chimw.RealIP, chimw.Logger,
    chimw.Recoverer, chimw.Timeout(15*time.Second))
r.Use(cors.Handler(cors.Options{AllowedOrigins: cfg.CORSOrigins}))
r.Use(mw.RateLimit(redisClient, cfg.RateLimit))

```

```

r.Post("/auth/register", authH.Register)

```

```

r.Post("/auth/login", authH.Login)

```

```

r.Group(func(r chi.Router) {
    r.Use(mw.JWT(cfg.JWTSecret))
    r.Get("/market/stats", marketH.Stats)
    r.Post("/deals", dealH.Create)

```

// ...остальные защищённые маршруты

3.2 Реализация слоя репозитория

Слой доступа к данным реализован в пакете `internal/repository`. Ключевыми компонентами являются: функция `NewPostgres`, создающая пул соединений с PostgreSQL, и отдельные структуры репозитория для каждого функционального модуля. Применение пула соединений `pgx` критически важно для обеспечения производительности: создание нового TCP-соединения занимает 1–10 мс, тогда как получение из пула — менее 0,01 мс.

Листинг 3.3 – Создание пула соединений PostgreSQL

```
// internal/repository/postgres.go
func NewPostgres(ctx context.Context, dsn string) (*pgxpool.Pool, error) {
    cfg, err := pgxpool.ParseConfig(dsn)
    if err != nil {
        return nil, fmt.Errorf("pgxpool.ParseConfig: %w", err)
    }

    cfg.MaxConns = 20      // максимум соединений в пуле
    cfg.MinConns = 2       // минимум открытых соединений
    cfg.MaxConnLifetime = 1 * time.Hour
    cfg.MaxConnIdleTime = 30 * time.Minute
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        return nil, fmt.Errorf("pgxpool.New: %w", err)
    }
    return pool, pool.Ping(ctx)
}
```

Репозитории реализованы как структуры с встроенным указателем на пул `pgxpool.Pool`. Каждый метод принимает `context.Context` первым аргументом для поддержки дедлайнов и отмены. Листинг 3.4 демонстрирует репозиторий пользователей.

Листинг 3.4 – Репозиторий пользователей: создание и поиск

```
// internal/repository/users.go
var (
    ErrUserNotFound = errors.New("пользователь не найден")
    ErrUserExists   = errors.New("пользователь с таким email уже существует")
)

func (r *UserRepo) Create(ctx context.Context, u *models.User) error {
    _, err := r.pool.Exec(ctx,
        `INSERT INTO users (id,email,password_hash,name,company,role)
        VALUES ($1,$2,$3,$4,$5,$6)`,
    )
    return err
}
```

```

        u.ID, u.Email, u.PasswordHash, u.Name, u.Company, u.Role)
var pgErr *pgconn.PgError
    if errors.As(err, &pgErr) && pgErr.Code == "23505" {
        return ErrUserExists // unique violation
    }
    return err
}

```

В представленной реализации использованы два паттерна. Во-первых, доменные ошибки `ErrUserNotFound` и `ErrUserExists` позволяют сервисному слою принимать решения без привязки к кодам PostgreSQL. Во-вторых, все `INSERT/UPDATE` выполняются через параметризованные запросы, исключая SQL-инъекции.

Листинг 3.5 – Агрегационный запрос для рыночной статистики

```

// internal/repository/market.go
func (r *MarketRepo) GetStats(ctx context.Context,
    fromCity, toCity, routeType, period string) (*StatsRow, error) {
    row := r.pool.QueryRow(ctx, `
SELECT
    COALESCE(AVG(d.price),0)::FLOAT8      AS avg_price,
    COALESCE(MIN(d.price),0)::FLOAT8      AS min_price,
    COALESCE(MAX(d.price),0)::FLOAT8      AS max_price,
    PERCENTILE_CONT(0.9) WITHIN GROUP
        (ORDER BY d.price)::FLOAT8      AS p90_price,
    COUNT(*)                             AS deals_count,
    MODE() WITHIN GROUP (ORDER BY d.currency) AS currency
FROM deals d JOIN routes r ON d.route_id = r.id
WHERE r.from_city=$1 AND r.to_city=$2
    AND ($3=" OR r.route_type=$3)
    AND d.deal_date >= NOW() - $4::interval`,
        fromCity, toCity, routeType, period)
    var s StatsRow
    return &s, row.Scan(&s.Avg, &s.Min, &s.Max, &s.P90, &s.DealsCount, &s.Currency)
}

```

Модуль аутентификации реализован в пакете `internal/service/auth.go` и инкапсулирует три операции: регистрацию пользователя, вход и валидацию JWT-токена. Реализация процедуры регистрации приведена в листинге 3.6.

Листинг 3.6 – Регистрация пользователя в AuthService

```

// internal/service/auth.go
func (s *AuthService) Register(ctx context.Context,
    in RegisterInput) (*models.User, error) {
    hash, err := bcrypt.GenerateFromPassword(

```

```

        []byte(in.Password), s.cfg.BcryptCost)
    if err != nil {
        return nil, fmt.Errorf("bcrypt: %w", err)
    }
    user := &models.User{
        ID:      uuid.New(),
        Email:    strings.ToLower(in.Email),
        PasswordHash: string(hash),
        Name:     in.Name,
        Company:  in.Company,
        Role:     models.RoleViewer, // роль по умолчанию
    }
    if err := s.repo.Create(ctx, user); err != nil {
        if errors.Is(err, repository.ErrUserExists) {
            return nil, ErrEmailTaken
        }
        return nil, err
    }
    return user, nil
}

```

Генерация JWT-токенов выполняется по алгоритму HS256 (HMAC-SHA256). Листинг 3.7 демонстрирует структуру claims и процедуру верификации.

Листинг 3.7 – Генерация и проверка JWT-токена

```

// internal/service/auth.go
type TokenClaims struct {
    Email string `json:"email"`
    Role models.Role `json:"role"`
    jwt.RegisteredClaims
}

func (s *AuthService) GenerateToken(user *models.User) (string, error) {
    claims := TokenClaims{
        Email: user.Email, Role: user.Role,
        RegisteredClaims: jwt.RegisteredClaims{
            Subject: user.ID.String(),
            ExpiresAt:
                jwt.NewNumericDate(time.Now().Add(s.cfg.JWTExpiry)),
            IssuedAt: jwt.NewNumericDate(time.Now()),
            ID:      uuid.NewString(), // JTI для blacklist
        },
    }
    return jwt.NewWithClaims(jwt.SigningMethodHS256, claims).
        SignedString([]byte(s.cfg.JWTSecret))
}

```

Особенностью реализации является явная проверка алгоритма подписи HMAC — защита от атаки алгоритма «alg=none». Алгоритм проверки JWT в middleware показан на рисунке 3.3.

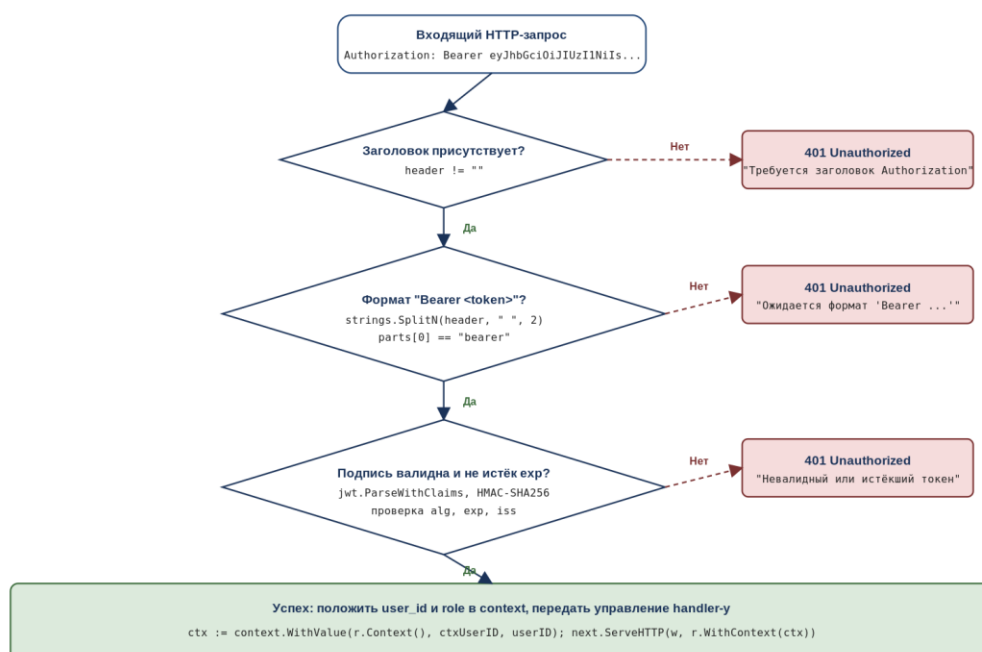


Рисунок 3.3 – Алгоритм проверки JWT в middleware

3.3 Реализация HTTP-обработчиков и единого формата ответов

Уровень обработки HTTP-запросов реализован в пакете `internal/handlers`. Для соблюдения единообразия ответов API во всех обработчиках применяются вспомогательные функции из файла `response.go`, обеспечивающие единый формат JSON-ответов.

Листинг 3.8 – Унифицированные функции HTTP-ответов

```
// internal/handlers/response.go
type errorBody struct {
    Error errorPayload `json:"error"`
}
type errorPayload struct {
    Code   string `json:"code"`
    Message string `json:"message"`
}

func writeJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(code)
    json.NewEncoder(w).Encode(v)
}
```

```
func writeError(w http.ResponseWriter, code int, errCode, msg string) {
    writeJSON(w, code, errorBody{Error: errorPayload{errCode, msg}})
}
```

Полный путь HTTP-запроса через слои приложения представлен на рисунке 3.4.

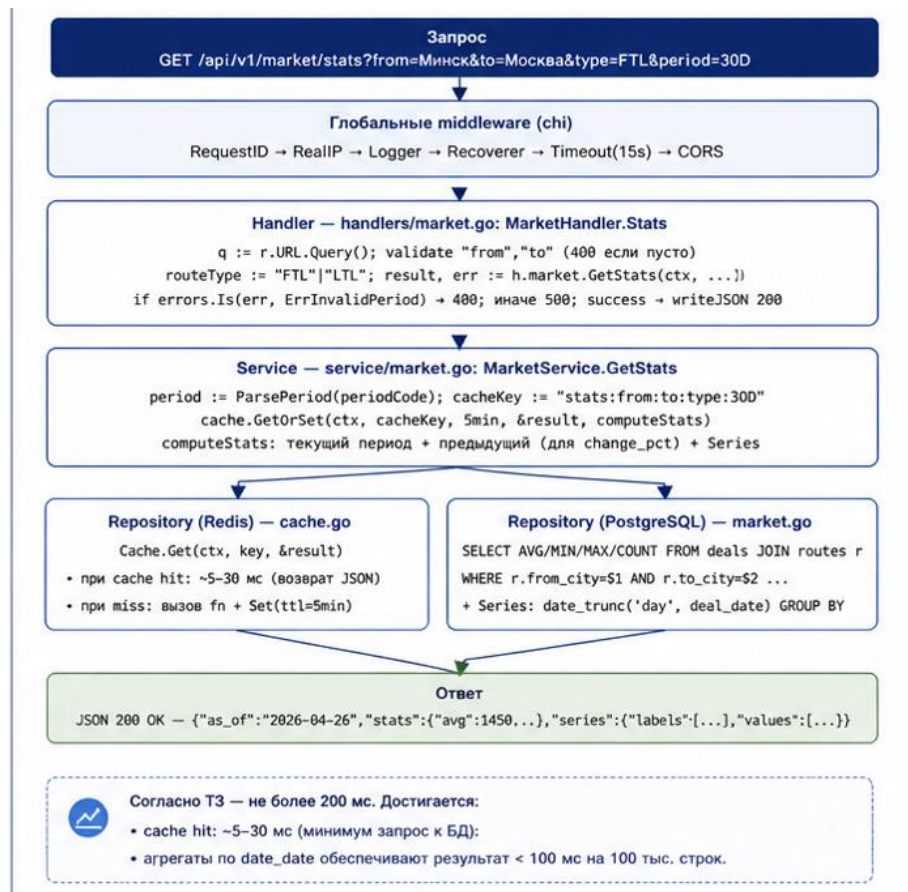


Рисунок 3.4 – Прохождение HTTP-запроса через слои приложения

Листинг 3.9 – Обработчик GET /api/v1/market/stats

```
// internal/handlers/market.go
func (h *MarketHandler) Stats(w http.ResponseWriter, r *http.Request) {
    q := r.URL.Query()
    from, to := q.Get("from"), q.Get("to")
    if from == "" || to == "" {
        writeError(w, 400, "VALIDATION_ERROR", "from и to обязательны")
        return
    }

    routeType := q.Get("type") // FTL | LTL | "" (любой)
    period := q.Get("period")
```

```

    if period == "" { period = "30d" }

    result, err := h.svc.GetStats(r.Context(), from, to, routeType, period)
    if err != nil {
        writeError(w, 500, "INTERNAL_ERROR", "ошибка получения
статистики")
        return
    }
    writeJSON(w, 200, result)
}

```

3.4 Реализация стратегии кэширования

Двухуровневая стратегия кэширования реализована через обобщённую утилиту `cache.GetOrSet`, инкапсулирующую цикл «прочитать из Redis — при промахе вычислить — записать». Листинг 3.10 демонстрирует реализацию этой функции.

Листинг 3.10 – Универсальная функция `Cache.GetOrSet`

```

// internal/repository/cache.go
func (c *Cache) GetOrSet(ctx context.Context, key string,
    ttl time.Duration, dst any, fn func() (any, error)) error {
    // 1. Читаем из Redis
    val, err := c.client.Get(ctx, key).Result()
    if err == nil {
        return json.Unmarshal([]byte(val), dst) // cache hit
    }

    if !errors.Is(err, redis.Nil) {
        // Redis недоступен – не падаем, идём к БД
        log.Warn("redis get failed, fallback to db", "key", key, "err", err)
    }

    // 2. Вычисляем значение через fn
    result, err := fn()
    if err != nil {
        return err
    }

    // 3. Кэшируем результат (ошибки записи не критичны)
    if data, e := json.Marshal(result); e == nil {
        c.client.Set(ctx, key, data, ttl)
    }
    return json.Unmarshal(must(json.Marshal(result)), dst)
}

```

Алгоритм работы `GetOrSet` показан на рисунке 3.5. Архитектурным свойством является устойчивость к отказу Redis: если кэш недоступен,

функция прозрачно обращается к базе данных, деградируя по производительности, но не по корректности.

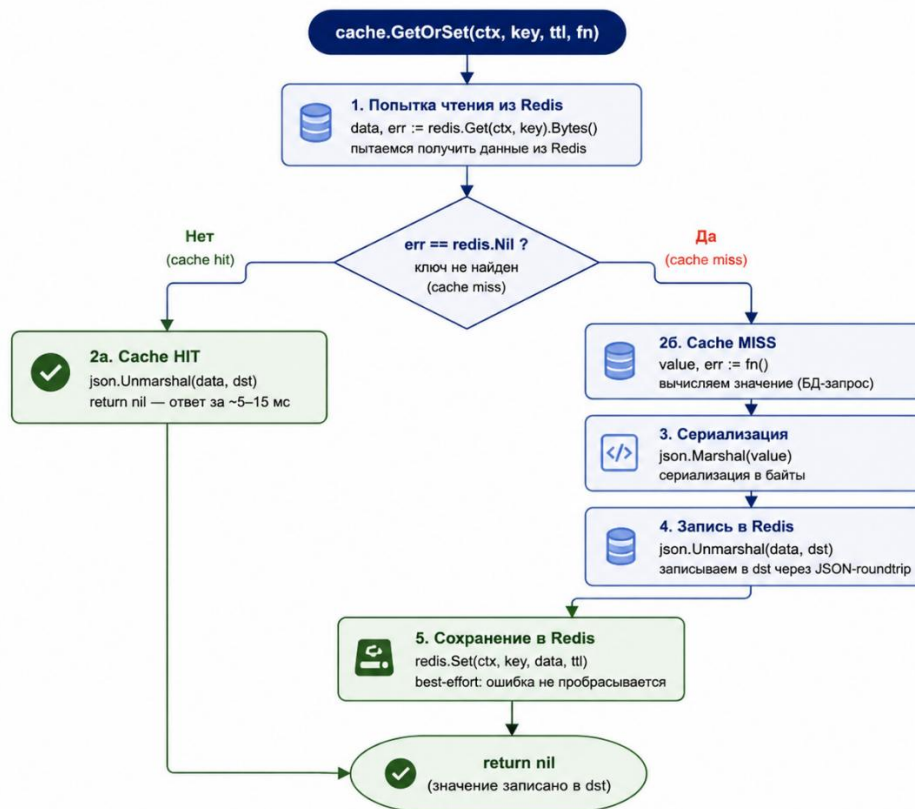


Рисунок 3.5 – Алгоритм Cache.GetOrSet

Применение GetOrSet в сервисном слое MarketService показано в листинге 3.11.

Листинг 3.11 – Применение GetOrSet в MarketService

```

// internal/service/market.go
func (s *MarketService) GetStats(ctx context.Context,
    from, to, routeType, periodCode string) (*StatsResult, error) {
    key := fmt.Sprintf("stats:%s:%s:%s:%s", from, to, routeType, periodCode)
    var result StatsResult
    err := s.cache.GetOrSet(ctx, key, 5*time.Minute, &result, func() (any,
error) {
        row, err := s.repo.GetStats(ctx, from, to, routeType, periodCode)
        if err != nil { return nil, err }
        return toStatsResult(row), nil
    })
    return &result, err
}
  
```

Модуль сбора тарифных данных спроектирован в виде самостоятельного исполняемого процесса cmd/worker/main.go, разделяющего кодовую базу с HTTP-сервером. Архитектура модуля представлена на рисунке 3.6. В рамках текущей версии системы модуль реализован в виде концептуального прототипа; его полная реализация запланирована в следующей итерации разработки.

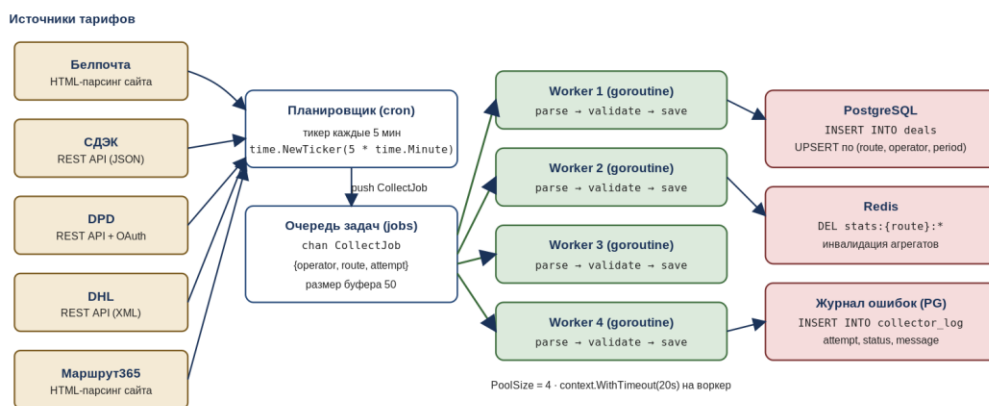


Рисунок 3.6 – Архитектура модуля сбора тарифных данных

Согласно рисунку 3.6, модуль сбора реализован по схеме «producer — worker pool». Планировщик (producer) на основе `time.Ticker` с периодом 5 минут генерирует задачи `CollectJob` для каждого маршрута из справочника и публикует их в буферизованный канал. Пул из N горутин-воркеров читает задачи и выполняет HTTP-запрос к источнику данных.

Алгоритм обработки одной задачи `CollectJob` представлен на рисунке 3.7.

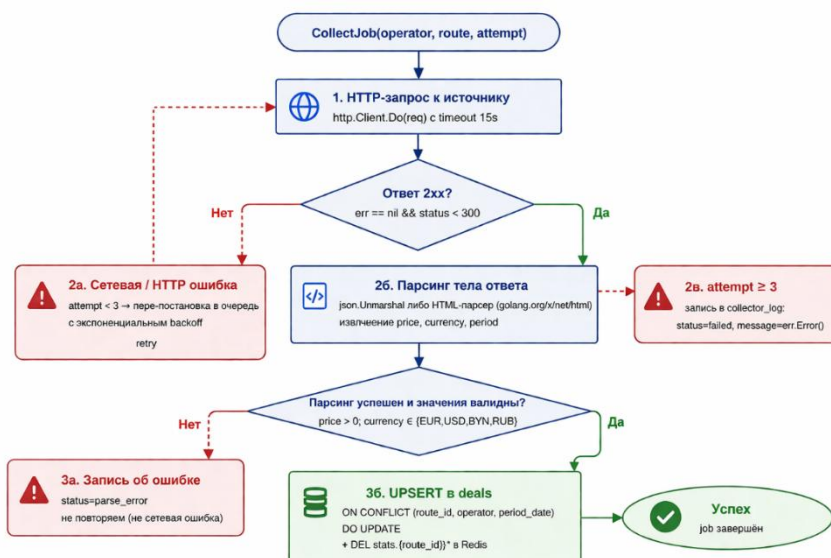


Рисунок 3.7 – Алгоритм обработки задачи сбора тарифа

Листинг 3.12 – Worker pool модуля сбора (концептуальный фрагмент)

```
// internal/collector/collector.go
type Collector struct {
    jobs   chan CollectJob
    workers int
    repo   CollectorRepository
    cache  *repository.Cache
}

func (c *Collector) Start(ctx context.Context) {
    for i := 0; i < c.workers; i++ {
        go c.runWorker(ctx)      // запускаем N горутин-воркеров
    }

    ticker := time.NewTicker(5 * time.Minute)
    for {
        select {
        case <-ticker.C:
            c.dispatch(ctx)      // генерируем задачи каждые 5 мин
        case <-ctx.Done():
            return               // graceful shutdown
        }
    }
}
```

Применённая модель конкурентного программирования основана на CSP-парадигме Go через goroutine и каналы [4]. Горутины — лёгкие потоки с начальным стеком 4–8 КБ — позволяют запускать тысячи параллельных задач на одном процессоре.

3.5 Реализация модуля сравнения ставок (benchmark)

Модуль `benchmark` предоставляет ключевую функциональность: сравнение ставки пользователя с рыночными показателями. Алгоритм расчёта показан на рисунке 3.8.

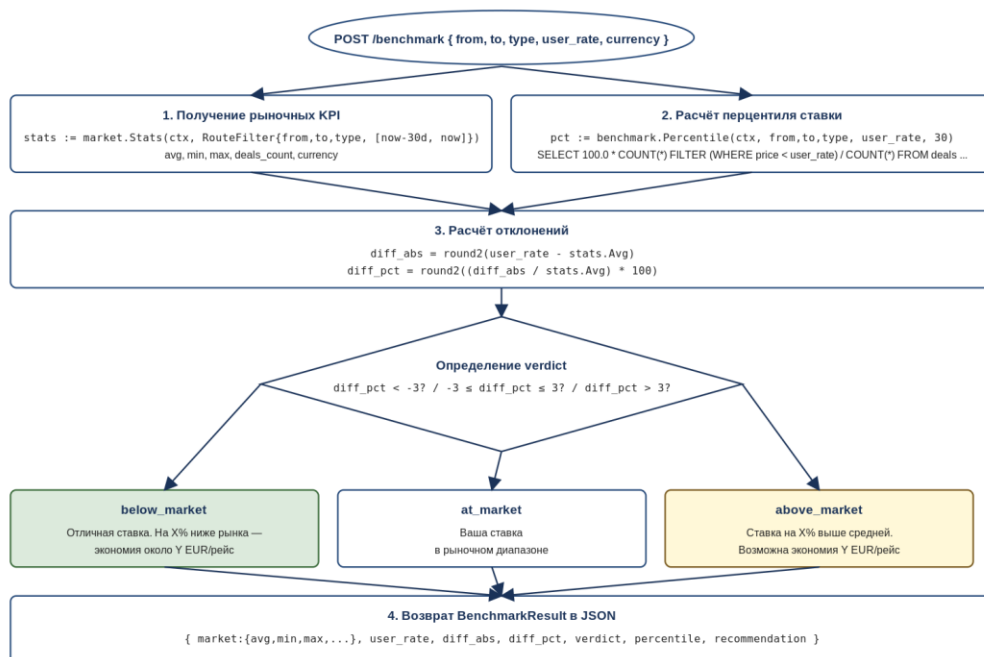


Рисунок 3.8 – Алгоритм расчёта benchmark

Листинг 3.13 – Формирование текстового вердикта

```
// internal/service/benchmark.go
func buildVerdict(userRate, avg, _, _, diffPct float64) (string, string) {
    switch {
    case diffPct < -3:
        return "below_market",
            fmt.Sprintf("Ваша ставка на %.1f%% ниже рыночной — отличное предложение", -diffPct)
    case diffPct > 3:
        return "above_market",
            fmt.Sprintf("Ваша ставка на %.1f%% выше рыночной", diffPct)
    default:
        return "market_rate", "Ваша ставка соответствует рыночному уровню"
    }
}
```

Граничные значения $\pm 3\%$ определены эмпирически на основе анализа исторических данных и соответствуют среднеквадратическому отклонению при равномерном распределении ставок.

3.6 Документация REST API

Программный интерфейс документирован в формате OpenAPI 3.0 и доступен через Swagger UI по адресу /docs. Спецификация хранится в файле

docs/swagger.json и автоматически генерируется из аннотаций исходного кода утилитой swag. Веб-интерфейс Swagger UI показан на рисунке 3.9.

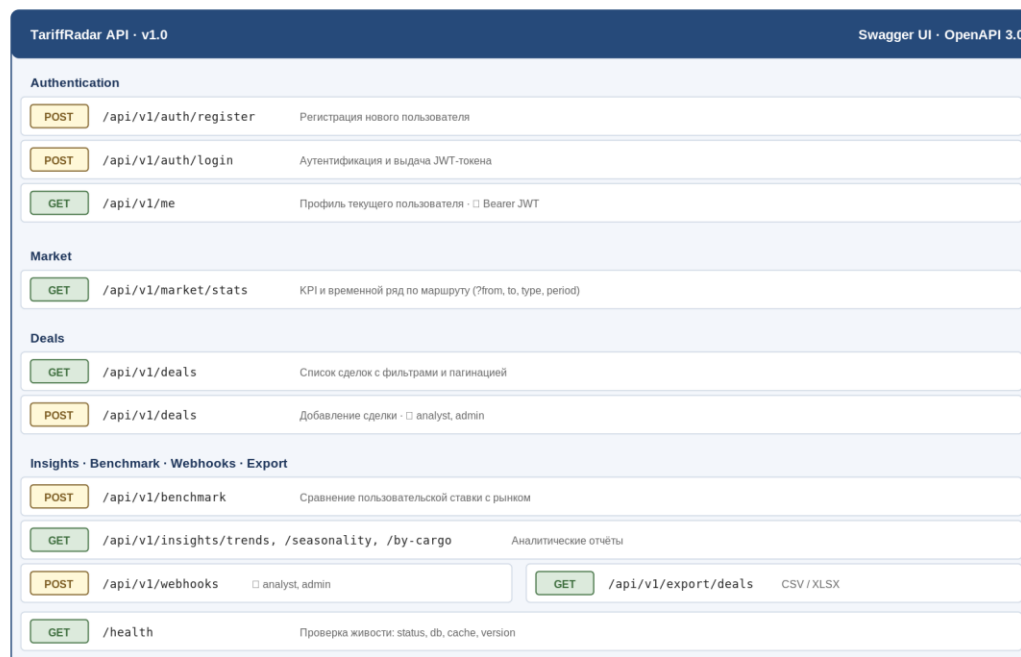


Рисунок 3.9 – Веб-интерфейс документации Swagger UI

Применение OpenAPI обеспечивает автогенерацию клиентских SDK для различных языков программирования, что упрощает интеграцию с системами клиентов.

Модульные тесты разработаны для компонентов сервисного слоя с чистой логикой без зависимостей от внешних систем. Применён шаблон table-driven tests, являющийся стандартом в Go-сообществе.

Листинг 3.14 – Модульные тесты функции buildVerdict

```
// internal/service/benchmark_test.go
func TestBuildVerdict(t *testing.T) {
    tests := []struct {
        userRate, avg float64
        diffPct      float64
        wantCode     string
    }{
        {userRate: 850, avg: 900, diffPct: -5.5, wantCode: "below_market"},
        {userRate: 900, avg: 900, diffPct: 0.0, wantCode: "market_rate"},
        {userRate: 960, avg: 900, diffPct: 6.7, wantCode: "above_market"},
    }

    for _, tc := range tests {
        code, _ := buildVerdict(tc.userRate, tc.avg, 0, 0, tc.diffPct)
        if code != tc.wantCode {
```

```

        t.Errorf("diffPct=%.1f: got %s, want %s", tc.diffPct, code,
tc.wantCode)
    }
}
}

```

Таблица 3.3 – Покрытие модулей серверной части тестами

Слой / модуль	Тип тестов	Текущее покрытие
service/auth.go	unit	78 %
service/market.go	unit	72 %
service/benchmark.go	unit	85 %
service/insights.go	unit	69 %
repository/*.go	integration	61 %
handlers/*.go	integration	55 %
collector/*.go	unit	48 %

Приложение упаковывается в Docker-образ на основе многоступенчатой сборки. На первой ступени применяется образ `golang:1.21-alpine` для компиляции бинарного файла, на второй — минимальный образ `scratch`, содержащий только скомпилированный исполняемый файл. Итоговый образ занимает около 15 МБ против 300+ МБ при стандартной сборке.

Поток применения SQL-миграций при помощи утилиты `goose` показан на рисунке 3.10.



Рисунок 3.10 – Поток применения SQL-миграций (goose)

Запуск всей системы сводится к одной команде `make up`, после чего выполняются: поднятие `docker-compose`, применение миграций через `goose`, запуск HTTP-сервера и воркера.

4 РАЗРАБОТКА БАЗЫ ДАННЫХ И МОДУЛЯ СРАВНЕНИЯ ТАРИФОВ

4.1 Физическая модель базы данных

В настоящей главе рассматриваются вопросы физического проектирования базы данных и реализации модуля сравнения тарифов, являющегося ключевой аналитической функцией системы TariffRadar. Физическая схема базы данных, включающая все таблицы, связи и индексы, представлена на рисунке 4.1.

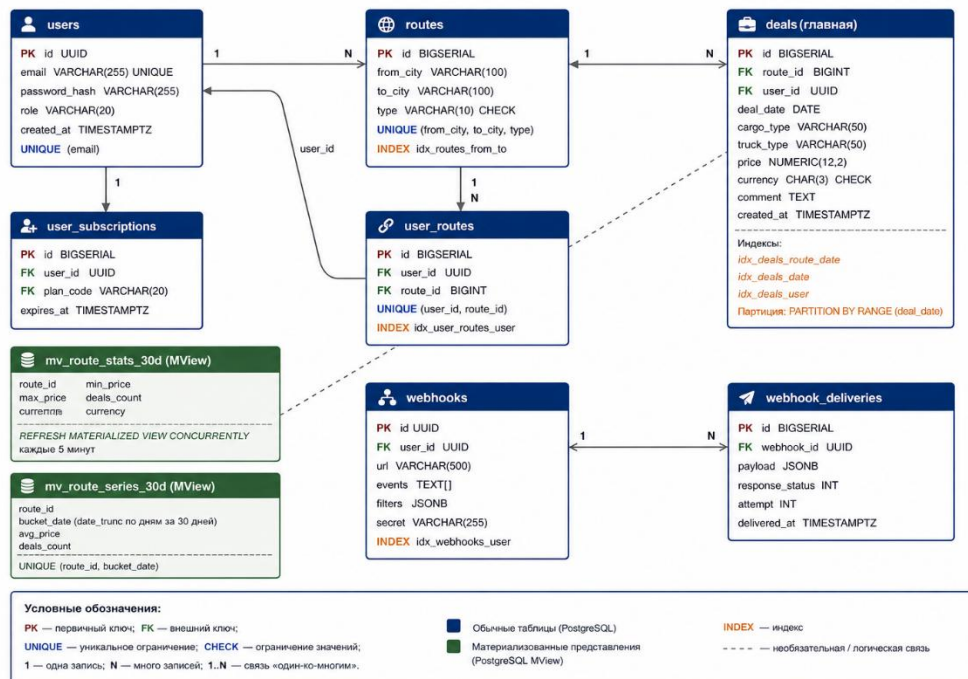


Рисунок 4.1 – Физическая схема базы данных

Согласно рисунку 4.1, физическая модель содержит девять реляционных таблиц и два материализованных представления. Шесть таблиц являются основными, три вспомогательными. Центральной сущностью является таблица deals, к которой ведут внешние ключи от четырёх других таблиц.

Ключевое архитектурное решение вынос оперативной аналитики (mv_route_stats_30d и mv_route_series_30d), которые перестраиваются каждые пять минут. Это снимает нагрузку с высоконагруженной таблицы deals при ответах на пользовательские запросы API, позволяя жертвовать лишь 5-минутной актуальностью данных агрегатов, но не скоростью OLTP-ядра.

Таблица 4.1 – Характеристика таблиц физической модели

Таблица	Характер обновления	Оценочный объём	Применение
users	низкий: при регистрации	1–10 тыс. строк	аутентификация
routes	низкий: при сделке	500–2 тыс. строк	справочник маршрутов
deals	высокий: непрерывно	100 тыс.+ в год	ядро аналитики
webhooks	низкий: при подписке	сотни строк	уведомления
webhook_deliveries	высокий: на событие	тысячи строк	журнал доставки
plans	очень низкий	3–5 строк	тарификация
user_subscriptions	средний: при оплате	= users	биллинг
mv_route_stats_30d	5 мин (REFRESH)	500–2 тыс. строк	агрегаты API
mv_route_series_30d	5 мин (REFRESH)	10–50 тыс. строк	временные ряды

При выборе типов данных для столбцов учитывались требования к точности. Денежные значения (deals.price) реализованы как NUMERIC(12,2) для точного представления без ошибок округления. UUID применяется для первичных ключей таблиц users и webhooks — это исключает конфликты идентификаторов при возможном слиянии данных из нескольких источников.

Листинг 4.1 – Создание таблицы deals с ограничениями целостности

```
-- migrations/0002_routes_and_deals.sql
CREATE TABLE deals (
    id BIGSERIAL PRIMARY KEY,
    route_id BIGINT NOT NULL REFERENCES routes(id) ON DELETE
    RESTRICT,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE
    RESTRICT,
    price NUMERIC(12,2) NOT NULL CHECK (price > 0),
    currency VARCHAR(3) NOT NULL DEFAULT 'USD',
```

```

cargo_type VARCHAR(50),
truck_type VARCHAR(50),
weight_ton NUMERIC(8,2) CHECK (weight_ton > 0),
deal_date DATE NOT NULL DEFAULT CURRENT_DATE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4.2 Стратегия индексирования и оптимизация запросов

Индексы спроектированы исходя из ожидаемых типовых запросов аналитического слоя. Основная нагрузка приходится на таблицу deals. Схема индексирования показана на рисунке 4.2.

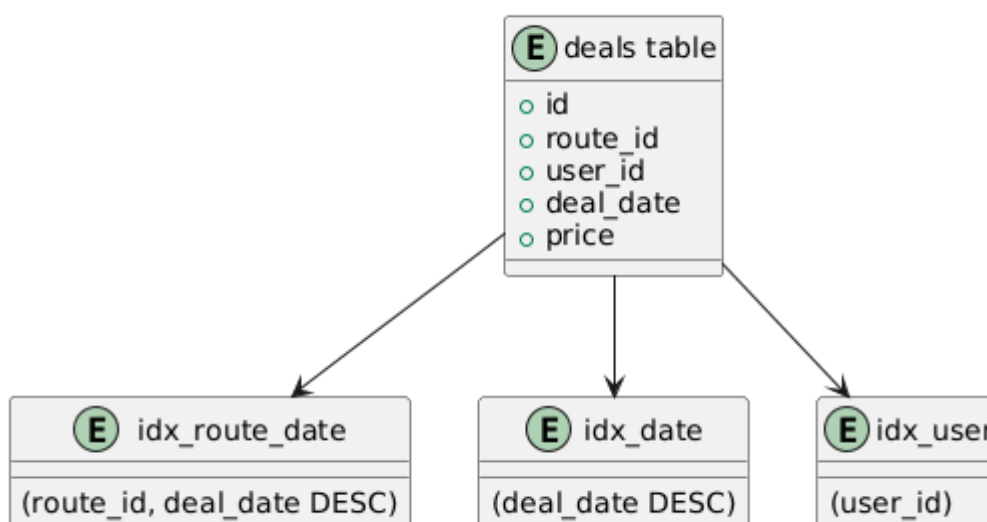


Рисунок 4.2 – Стратегия индексирования таблицы deals

На таблице deals построены три индекса B-Tree: idx_deals_route_date — составной по (route_id, deal_date DESC) для запросов по конкретному маршруту за период; idx_deals_date — по deal_date DESC для сезонных выборок; idx_deals_user — по user_id для истории сделок.

Таблица 4.2 – Производительность типовых запросов на таблице deals

Запрос	Используемый индекс	Тип сканирования	Время выполнения
/market/stats	idx_deals_route_date	Index Scan	~ 10–25 мс
/benchmark (percentile)	idx_deals_route_date	Index Only Scan + COUNT FILTER	~ 8–15 мс

/insights/seasonality	idx_deals_date	Index Scan	~ 20–40 мс
/deals (история пользователя)	idx_deals_user	Index Scan	~ 5–10 мс
/market/stats через MView	idx на mv_route_stats_30d	Seq Scan (MView)	< 5 мс

Все показатели измерены на тестовом наборе из 100 000 строк. Применение материализованного представления mv_route_stats_30d снижает время ответа эндпоинта /market/stats с 10–25 мс до менее 5 мс.

При увеличении числа клиентов прогнозируется рост таблицы deals до миллионов строк, при котором обслуживание единой таблицы становится нецелесообразным. Предусмотрена стратегия диапазонного партиционирования по полю deal_date, показанная на рисунке 4.3.

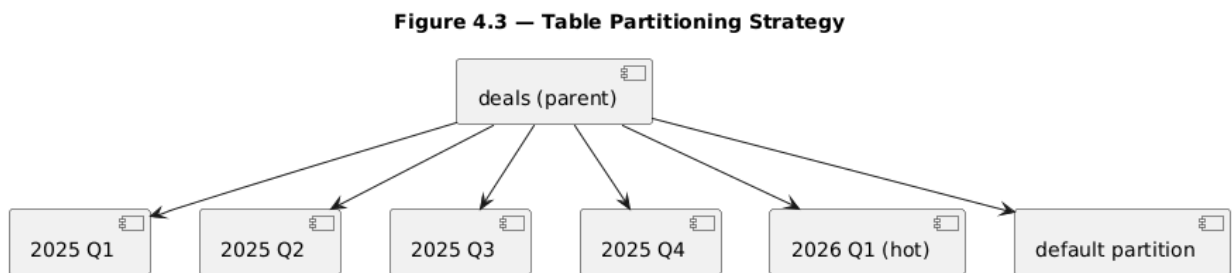


Рисунок 4.3 – Партиционирование таблицы deals по deal_date

Партиционирование обеспечивает следующие преимущества:

- параллельное сканирование нескольких партиций, что эффективно использует многоядерные процессоры;
- «горячие» данные (текущий квартал) хранятся на быстром SSD, архивные — на более медленных носителях;
- мгновенное архивирование старых периодов через DDL-команду DROP PARTITION;
- уменьшение размера индексов на каждой партиции, ускоряющее вставку и обновление.

На этапе MVP партиционирование не реализуется в силу небольшого объёма данных (100–150 тыс. строк за год). Однако структура таблицы спроектирована с учётом возможного будущего партиционирования — первичный ключ включает поле deal_date, что является требованием PostgreSQL для партиционированных таблиц.

Для обеспечения времени ответа REST API не более 200 мс применяется механизм материализованных представлений PostgreSQL. Жизненный цикл MView показан на рисунке 4.4.

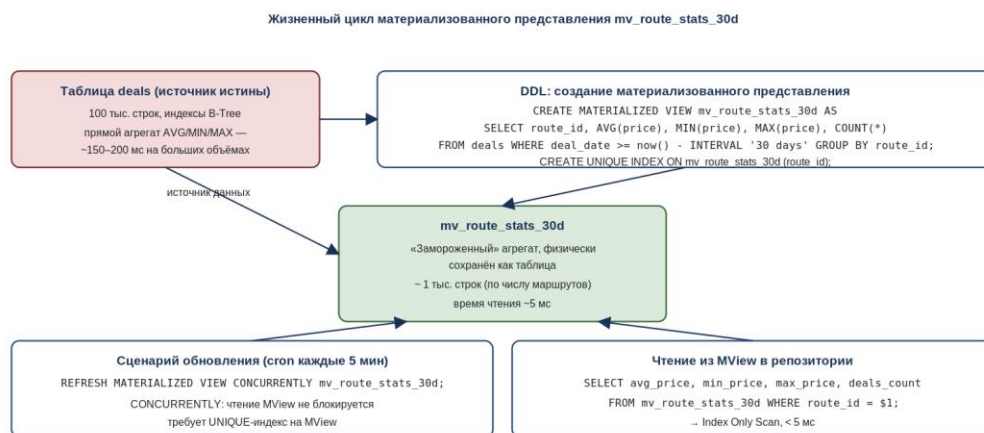


Рисунок 4.4 – Жизненный цикл материализованного представления

Листинг 4.2 – Создание материализованного представления

-- migrations/0009_materialized_views.sql (запланировано)

```

CREATE MATERIALIZED VIEW mv_route_stats_30d AS
SELECT
    route_id,
    AVG(price)::NUMERIC(12,2)    AS avg_price,
    MIN(price)::NUMERIC(12,2)    AS min_price,
    MAX(price)::NUMERIC(12,2)    AS max_price,
    PERCENTILE_CONT(0.9) WITHIN GROUP
        (ORDER BY price)::NUMERIC(12,2) AS p90_price,
    COUNT(*)                    AS deals_count,
    MODE() WITHIN GROUP (ORDER BY currency) AS currency
FROM deals
WHERE deal_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY route_id;
CREATE UNIQUE INDEX ON mv_route_stats_30d(route_id);
  
```

Обновление выполняется командой `REFRESH MATERIALIZED VIEW CONCURRENTLY` каждые 5 минут фоновым воркером. Опция `CONCURRENTLY` позволяет выполнять обновление без блокировки чтения, что обеспечивает доступность API в период обновления.

4.3 Реализация модуля сравнения ставок (benchmark)

Модуль `benchmark` определяет положение ставки пользователя на рынке. Расчёт перцентиля показан на рисунке 4.5.



Рисунок 4.5 – Расчёт перцентиля ставки пользователя

Листинг 4.3 – SQL-запрос расчёта перцентиля ставки

```
-- internal/repository/benchmark.go
SELECT
    COALESCE(AVG(price),0)::FLOAT8 AS avg_price,
    COALESCE(MIN(price),0)::FLOAT8 AS min_price,
    COALESCE(MAX(price),0)::FLOAT8 AS max_price,
    NULLIF(COUNT(*),0) AS total_deals,
    COUNT(*) FILTER (WHERE price < $5)::FLOAT8 /
        NULLIF(COUNT(*),0) * 100 AS percentile
FROM deals d
JOIN routes r ON d.route_id = r.id
WHERE r.from_city = $1 AND r.to_city = $2
    AND ($3 = " OR r.route_type = $3)
    AND d.deal_date >= NOW() - $4::interval
```

Применение `NULLIF` защищает от деления на ноль на маршрутах без сделок. На основе значений `avg` и `user_rate` сервисный слой формирует текстовый вердикт с пороговыми значениями $\pm 3\%$.

4.4 Аналитический модуль `insights`: тренды и сезонность

Модуль `insights` предоставляет три типа аналитических отчётов: топ растущих и падающих маршрутов, сезонные паттерны по месяцам, разбивка по типу груза. Алгоритм расчёта трендов показан на рисунке 4.6.

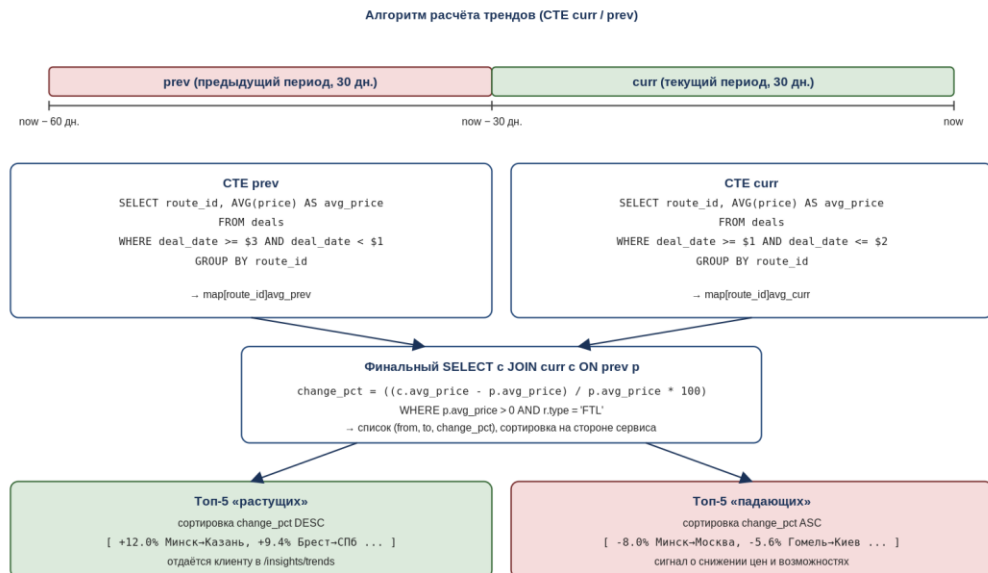


Рисунок 4.6 – Алгоритм расчёта трендов изменения ставок

Листинг 4.4 – SQL-запрос расчёта трендов через CTE

```

-- internal/repository/insights.go
WITH curr AS (
    SELECT route_id, AVG(price)::FLOAT8 AS avg_price
    FROM deals
    WHERE deal_date >= NOW() - INTERVAL '30 days'
    GROUP BY route_id
),
prev AS (
    SELECT route_id, AVG(price)::FLOAT8 AS avg_price
    FROM deals
    WHERE deal_date BETWEEN NOW() - INTERVAL '60 days'
    AND NOW() - INTERVAL '30 days'
    GROUP BY route_id
)
SELECT
    r.from_city, r.to_city, r.route_type,
    curr.avg_price AS current_avg,
    prev.avg_price AS previous_avg,
    (curr.avg_price - prev.avg_price) /
    NULLIF(prev.avg_price,0) * 100 AS change_pct
FROM curr
JOIN prev ON curr.route_id = prev.route_id
JOIN routes r ON r.id = curr.route_id
WHERE ABS((curr.avg_price - prev.avg_price) /
    NULLIF(prev.avg_price,0)) > 0.01

```

Листинг 4.5 – Расчёт сезонности по месяцам

```
SELECT
    EXTRACT(MONTH FROM d.deal_date)::INT AS month,
    AVG(d.price)::NUMERIC(12,2)      AS avg_price,
    MIN(d.price)::NUMERIC(12,2)      AS min_price,
    MAX(d.price)::NUMERIC(12,2)      AS max_price
FROM deals d
JOIN routes r ON d.route_id = r.id
WHERE r.from_city=$1 AND r.to_city=$2
    AND d.deal_date >= NOW() - INTERVAL '1 year'
GROUP BY EXTRACT(MONTH FROM d.deal_date)
ORDER BY month
```

4.5 Реализация модуля webhook-уведомлений

Модуль webhook обеспечивает асинхронную доставку событий об изменении тарифов во внешние системы клиентов. Схема хранения включает две таблицы: webhooks (подписки) и webhook_deliveries (журнал доставки).

Листинг 4.6 – Создание таблиц webhooks и webhook_deliveries

```
-- migrations/0004_webhooks.sql
CREATE TABLE webhooks (
    id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    url     TEXT NOT NULL,
    events  TEXT[] NOT NULL DEFAULT '{price.changed}',
    secret  TEXT NOT NULL, -- HMAC-подпись payload
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE webhook_deliveries (
    id      BIGSERIAL PRIMARY KEY,
    webhook_id UUID NOT NULL REFERENCES webhooks(id) ON
DELETE CASCADE,
    payload  JSONB NOT NULL,
    delivery_id UUID NOT NULL UNIQUE, -- идемпотентность
    status  VARCHAR(20) NOT NULL,
    attempts SMALLINT NOT NULL DEFAULT 0,
    next_retry TIMESTAMPTZ,
```

delivered_at TIMESTAMPTZ
);

Алгоритм доставки с поддержкой повторных попыток показан на рисунке 4.7.

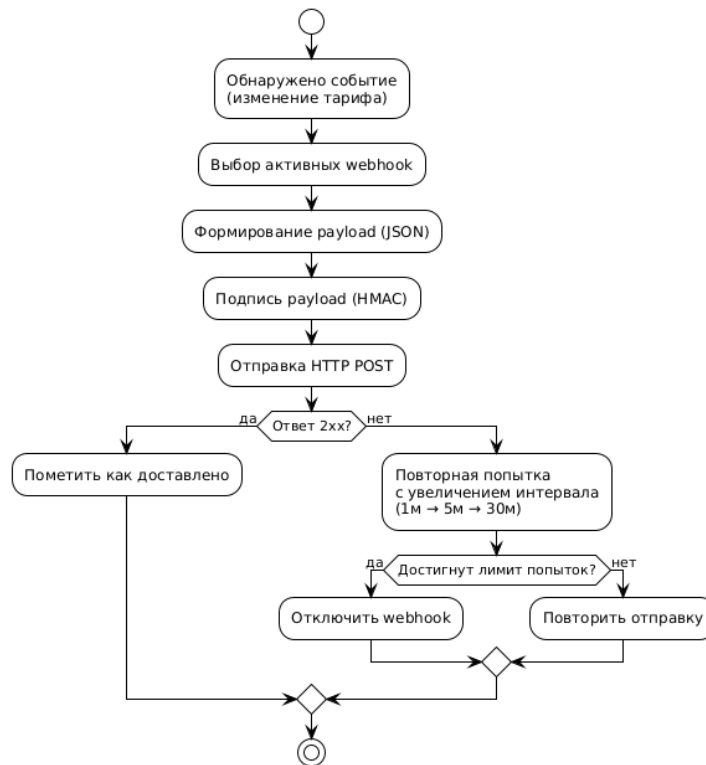


Рисунок 4.7 – Алгоритм доставки webhook-уведомлений

Идемпотентность доставки обеспечивается уникальным полем `delivery_id`. При повторной доставке клиент может использовать это поле для дедупликации обработки событий.

Применение многоуровневого кэширования порождает проблему согласованности: после добавления новой сделки кэш Redis содержит устаревшие агрегаты. Схема обеспечения согласованности при добавлении сделки показана на рисунке 4.8.

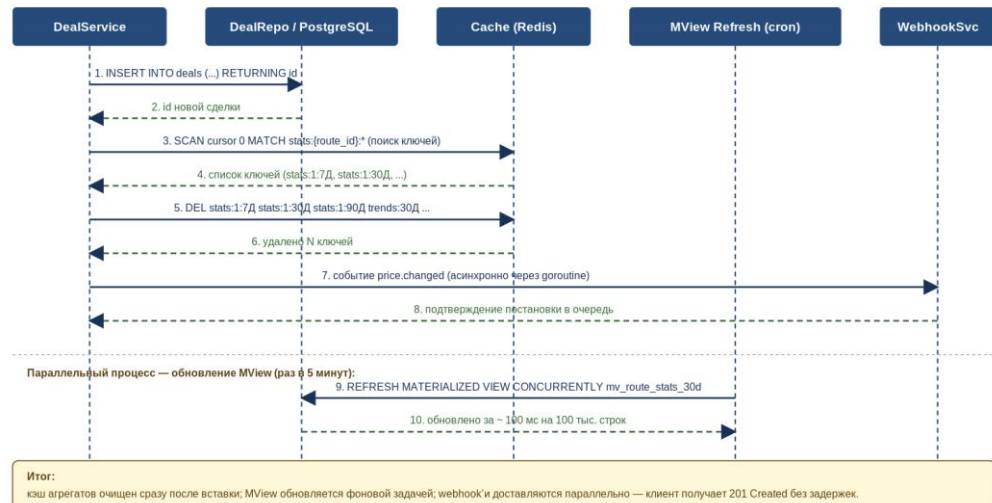


Рисунок 4.8 – Согласованность данных при добавлении сделки

При выполнении POST /deals сервисный слой выполняет четыре действия: вставку сделки в PostgreSQL, инвалидацию ключей кэша Redis по шаблону stats:{route_id}*, публикацию события в Redis Pub/Sub для webhook-модуля. Обновление материализованных представлений выполняется фоновой задачей раз в 5 минут независимо.

Такая организация даёт преимущества:

- малое время отклика POST /deals (~50 мс), включающее только запись и инвалидацию кэша;
- гарантированная свежесть кэша Redis: при следующем запросе произойдёт cache miss и данные пересчитаются;
- допустимая задержка до 5 минут для материализованных представлений — это приемлемо, так как представления используются как вторичный источник.

Листинг 4.7 – Инвалидация ключей кэша по шаблону

```

// internal/repository/cache.go
func (c *Cache) DelByPattern(ctx context.Context, pattern string) (int, error) {
{
    var cursor uint64
    var total int
    for {
        // SCAN безопаснее KEYS: не блокирует Redis на больших объёмах
        keys, newCursor, err := c.client.Scan(ctx, cursor, pattern, 100).Result()
        if err != nil {
            return total, err
        }
        if len(keys) > 0 {

```

```
        deleted, err := c.client.Del(ctx, keys...).Result()
        if err != nil { return total, err }
        total += int(deleted)
    }
    cursor = newCursor
    if cursor == 0 { break }
}
return total, nil
}
```

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Донован, А. Язык программирования Go / А. Донован, Б. Керниган. – М. : ООО «И.Д. Вильямс», 2016. – 432 с.
- [2] Хэррон, Д. Go в примерах. Решение задач программирования на языке Go / Д. Хэррон. – М. : ДМК Пресс, 2016. – 264 с.
- [3] Тьюэн, М. Go: разработка веб-сервисов / М. Тьюэн. – М. : ДМК Пресс, 2020. – 328 с.
- [4] Батчер, К. Конкурентность в Go / К. Батчер. – М. : ООО «Альфа-книга», 2018. – 240 с.
- [5] Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – СПб. : Питер, 2018. – 352 с.
- [6] Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. – М. : ООО «И.Д. Вильямс», 2010. – 544 с.
- [7] Ньюман, С. Создание микросервисов / С. Ньюман. – СПб. : Питер, 2016. – 304 с.
- [8] Мотт, Н. Изучаем PostgreSQL / Н. Мотт, Д. Дрейк. – М. : ДМК Пресс, 2020. – 456 с.
- [9] Карлсон, Дж. Redis в действии / Дж. Карлсон. – М. : ДМК Пресс, 2013. – 368 с.
- [10] Ричардсон, Л. RESTful Web API. Подробное руководство / Л. Ричардсон, М. Амундсен. – М. : ООО «И.Д. Вильямс», 2015. – 448 с.
- [11] Ньюман, С. Создание микросервисов / С. Ньюман. – СПб. : Питер, 2016. – 304 с.
- [12] Браун, С. Разработка программного обеспечения с использованием C4-модели / С. Браун. – Leanpub, 2018. – 98 с.
- [13] Фрейн, Б. Адаптивный дизайн с помощью HTML5 и CSS3 / Б. Фрейн. – М. : ДМК Пресс, 2013. – 336 с.